Универзитет у Крагујевцу

Bogdan Babaev

Forecasting Time-Varying Intermarket Dependencies Between Cryptocurrencies and Conventional Assets Using Machine Learning

Мастер рад

Крагујевац, 2026.

Универзитет у Крагујевцу

Назив студијског програма: ВЕШТАЧКА ИНТЕЛИГЕНЦИЈА

Ниво студија: Мастер академске студије

Предмет: Напредно машинско учење

Број индекса: 1ВИ/2023

Bogdan Babaev

Forecasting Time-Varying Intermarket Dependencies Between Cryptocurrencies and Conventional Assets Using Machine Learning

Мастер рад

Комисија за преглед и одбрану:

1. **проф. др Владимир М. Миловановић - ментор**
2. **TODO: члан комисије 1**
3. **TODO: члан комисије 2**

Датум одбране: _____

Оцена: _____

У оквиру овог мастер рада кандидат треба да истражи могућност прогнозирања временски-променљивих међутржишних зависности између криптовалута и конвенционалних финансијских актива применом модела машинског учења. У склопу тога кандидат треба да:

1. прикупи историјске дневне цене за Bitcoin и традиционалне активе (акцијске индексе, фондове племенитих метала, индекс америчког долара, Ethereum) и изврши одговарајућу предобраду и синхронизацију скупа података;
2. развије репродуцибилан рачунарски поступак за израчунавање временски-променљивих корелационих циљева помоћу клизних прозора различитих дужина и конструкцију предиктивних особина заснованих на инерцији зависности, волатилности и повратима;
3. реализује ванузорачну евалуацију и поређење неколико модела машинског учења (AR, ElasticNet, Ridge, Random Forest, GBM, XGBoost) са DCC-GARCH(1,1) економетријским репером применом процедуре проклизавајућег прозора и Diebold-Mariano теста статистичке значајности;
4. уведе практични сигнални слој за рано детектовање стресних дана на тржишту традиционалних актива на основу динамике криптовалута и прогноза зависности, и вреднује га метрикама класификационе тачности прикладним за неуравнотежен скуп класа.

Препоручена литература:

[0] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models", у: *Journal of Business & Economic Statistics* 20.3 (2002.), стр. 339–350.

[0] Francis X. Diebold и Roberto S. Mariano, „Comparing Predictive Accuracy", у: *Journal of Business & Economic Statistics* 13.3 (1995.), стр. 253–263.

[0] Hui Zou и Trevor Hastie, „Regularization and Variable Selection via the Elastic Net", у: *Journal of the Royal Statistical Society: Series B* 67.2 (2005.), стр. 301–320.

[0] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine", у: *The Annals of Statistics* 29.5 (2001.), стр. 1189–1232.

[0] Tianqi Chen и Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System", у: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016., стр. 785–794.

Крагујевац, 19. 5. 2026.

Ментор:

Др Владимир М. Миловановић,
редовни професор

_____

Abstract

This thesis investigates whether information contained in cryptocurrency dynamics can be used to forecast time-varying dependency structures with conventional assets and, in a second step, to derive an early warning signal for investors. Bitcoin is treated as the base cryptocurrency, while the conventional asset universe includes equity indices, precious-metal exchange-traded funds, the U.S. dollar index, and Ethereum as an additional crypto reference. A reproducible machine-learning pipeline is developed for data acquisition, return construction, rolling-correlation target generation, feature engineering, walk-forward model evaluation, and benchmarking against a leakage-safe DCC-GARCH specification. The empirical results show that intermarket dependency is forecastable, but that most predictive power originates from persistence in the dependency series itself. Consequently, simple autoregressive baselines outperform more complex models on average, while machine-learning models still provide useful improvements relative to the DCC-GARCH benchmark. An investor-oriented signal layer based on crypto features and dependency forecasts achieves moderate but meaningful performance and is best interpreted as a risk-management overlay rather than a standalone market-timing engine.

Keywords: cryptocurrencies, Bitcoin, intermarket dependency, correlation, machine learning, DCC-GARCH, forecasting, risk management.

Резиме

У овом мастер раду разматра се проблем прогнозирања променљивих међутржишних зависности између криптовалута и традиционалних финансијских актива. У фокусу је Bitcoin као основна криптовалута, док скуп традиционалних актива обухвата акцијске индексе, фондове везане за племените метале и индекс америчког долара. Развијен је репродуцибилан рачунарски поступак који покрива преузимање података, израчунавање логаритамских приноса, формирање временски променљивих корелационих циљева, конструисање особина, ванузорачну процену више модела машинског учења и поређење са DCC-GARCH економетријским репером. Поред основног задатка прогнозирања зависности, уведен је и практични слој за формирање раног сигнала ризика који треба да упозори инвеститора на стресне дане на тржишту традиционалних актива. Резултати показују да је временски променљива међутржишна зависност предвидива, али да највећи део предвидивости потиче из сопствене инерције циљне серије, док сложенији модели машинског учења доносе ограничено додатно побољшање у односу на једноставне ауторегресионе приступе. Практични сигнални слој даје умерене резултате и има смисла пре свега као додатни филтар ризика, а не као самосталан механизам за свакодневно одређивање смера тржишта.

Кључне речи: криптовалуте, Bitcoin, међутржишне зависности, корелација, машинско учење, DCC-GARCH, управљање ризиком.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full form
AR1	First-order autoregressive model
AUC	Area under the ROC curve
BTC	Bitcoin (`BTC-USD`)
CSV	Comma-separated values
DCC	Dynamic Conditional Correlation
DCC-GARCH	Dynamic Conditional Correlation GARCH(1,1)
DM	Diebold–Mariano (test)
EDA	Exploratory Data Analysis
ElasticNet	Elastic Net regularized regression
ETF	Exchange-Traded Fund
ETH	Ethereum (`ETH-USD`)
F1	F1 score (harmonic mean of precision and recall)
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
GBM	Gradient Boosting Machine
GLD	SPDR Gold Shares ETF (`GLD`)
GPU	Graphics Processing Unit
GSPC	S&P 500 index (`^GSPC`)
HAR	Heterogeneous Autoregressive (model)
IXIC	NASDAQ Composite index (`^IXIC`)
MAE	Mean Absolute Error
ML	Machine Learning
NASDAQ	National Association of Securities Dealers Automated Quotations
OOS	Out-of-sample
RF	Random Forest
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
S&P 500	Standard & Poor's 500 index
SLV	iShares Silver Trust ETF (`SLV`)
UUP	Invesco DB US Dollar Index Bullish Fund (`UUP`)
XGB	XGBoost (Extreme Gradient Boosting)

1 Introduction

1.1 Motivation and research context

Over the last decade, Bitcoin went from a curiosity on niche internet forums to an asset class that institutional investors actually worry about. That shift has a concrete consequence for risk management: when a previously isolated market grows large enough to attract serious capital, its price swings start to matter for portfolios that never held a single satoshi. The question is not whether Bitcoin now influences other markets, but *how* and *when* that influence changes over time.

This thesis focuses on *time-varying intermarket dependency*—the evolving statistical relationship between Bitcoin and conventional assets like equity indices, gold, silver, and the dollar. Rather than asking whether Bitcoin and the S&P 500 are correlated on average, the question is whether that correlation is predictable at all, and whether cryptocurrency price dynamics contain any early-warning signal for stress in traditional markets.

The practical angle matters. A risk manager does not need a crystal-ball return forecast to get value from this kind of analysis. What matters is recognising when co-movement patterns are changing, when a market regime is shifting from decorrelated to tightly coupled, or when the probability of a bad day on conventional markets is quietly rising. Even a modest, statistically honest signal of that kind can be useful—as long as you know what it can and cannot do.

1.2 Research questions

The thesis is structured around two connected layers. The first is a forecasting problem: can rolling correlation between Bitcoin and a conventional asset be predicted one step ahead, and how do machine-learning models compare to a classical econometric benchmark under strict out-of-sample evaluation? The second is an applied question: can those forecasts be turned into a practical warning signal for investors?

More specifically, the four research questions addressed are:

1. Is the rolling dependency between Bitcoin and conventional assets forecastable out of sample?
2. Do machine-learning models outperform a leakage-safe DCC-GARCH benchmark?
3. Do machine-learning models add value beyond simple persistence-based baselines such as AR1?
4. Can crypto-derived dependency forecasts support a practically useful stress-warning signal for conventional markets?

1.3 Contributions of the thesis

To address these questions, a reproducible Python pipeline was built from scratch. It downloads price data, constructs log-return series, computes rolling correlations across multiple window lengths, engineers lagged and volatility-based features, and evaluates a range of models within a walk-forward experimental design. The model set includes

two naive baselines (random-walk and AR(1)), regularised linear models (Ridge, Elastic-Net), tree-based ensembles (Random Forest, GBM, XGBoost), and a leakage-safe DCC-GARCH(1,1) benchmark. On top of the forecasting stage, a second layer converts predictions into a binary stress classifier for conventional assets.

The main empirical finding is that the dependency process is forecastable out of sample—but that most of that predictability comes from its own past, not from novel cross-asset signals. Autoregressive baselines dominate the average performance ranking. This is not a failure; it is a precise characterisation of what the forecasting problem actually looks like. Machine learning adds value relative to DCC-GARCH and helps most in the stress-classification task, where the problem structure suits ensemble methods better than pure time-series regression.

The contribution is threefold: (i) a clean, reproducible framework for rolling intermarket dependency forecasting; (ii) a systematic comparison of ML and econometric methods under consistent walk-forward evaluation; and (iii) an explicit bridge from statistical forecasting to a practically relevant investor-warning application.

1.4 Thesis outline

Section 2 reviews the relevant theory and literature: time-varying dependence, financial contagion, machine-learning forecasting methods, and the Diebold–Mariano test. Section 3 describes the data, target construction, feature engineering, model specifications, and evaluation design. Section 4 reports empirical results for both layers. Section 5 interprets and qualifies those results. Section 6 concludes.

2 Theoretical Background and Literature Review

2.1 Why intermarket dependency matters

Intermarket dependency occupies a central position in modern financial theory because the benefits of diversification, the propagation of contagion, the efficacy of hedging strategies, and the measurement of portfolio risk all depend fundamentally on the co-movement structure among assets. In the canonical mean-variance framework, dependence is summarized by a constant correlation coefficient. Empirical evidence, however, has long established that this static representation is inadequate. Pairwise correlations are time-varying, respond asymmetrically to positive and negative shocks, and tend to rise sharply during market downturns—precisely the episodes in which diversification is most urgently required [0]. Consequently, the forecasting of *time-varying dependence* is both theoretically motivated and practically indispensable for risk management.

The relevance of dynamic dependence is amplified when one of the assets under study is cryptocurrency-based. Bitcoin trades on a continuous, globally accessible market, responds rapidly to shifts in sentiment and funding liquidity, and is simultaneously subject to a broad range of economic narratives: inflation-hedging demand, speculative risk appetite, technological optimism, regulatory uncertainty, and global macro positioning. This multiplicity of drivers means that Bitcoin occupies no single, stable role in a multi-asset portfolio. At different points in the business and credit cycle, it may exhibit the characteristics of a speculative high-beta growth instrument, a liquidity-sensitive risk proxy, or an idiosyncratic sentiment barometer. That structural instability renders its relationship with conventional assets uniquely suitable for a time-varying analytical approach.

2.2 Cryptocurrencies and conventional assets

The academic literature on the intersection of cryptocurrency markets and conventional asset classes has expanded substantially over the past decade, developing along several distinct research strands. One strand examines Bitcoin as a standalone asset and characterizes its return distribution, volatility clustering, tail risk profile, and degree of weak-form informational efficiency. A second strand investigates portfolio diversification and asks whether Bitcoin provides economically meaningful diversification benefits relative to equity indices, gold, crude oil, or major fiat currencies. A third strand, most directly relevant to the present thesis, studies connectedness, contagion, and return and volatility spillovers across markets, with particular attention to crisis and stress episodes.

A recurring finding across these strands is structural instability [0, 0]. The dependence between Bitcoin and broad equity indices tends to be statistically weak or directionally inconsistent during tranquil market conditions but strengthens materially during episodes of pronounced risk-on or risk-off positioning [0, 0]. Precious metals exhibit generally lower and more variable co-movement with Bitcoin [0, 0], a pattern attributable to their distinct investor bases and established safe-haven role. Currency-related instruments introduce a further dimension, as they serve as proxies for global funding liquidity [0].

These empirical regularities motivate the modeling strategy adopted in the present thesis. Rather than imposing a static dependence coefficient or estimating a single full-sample correlation, the thesis models dependence as a time-varying process. This specification accommodates gradual regime drift, abrupt structural breaks, and heterogeneity in the

pace and direction of co-movement changes across window lengths and asset pairs.

2.3 Rolling correlation as a forecasting target

The rolling window correlation estimator is conceptually transparent and computationally tractable. For two return series $\{r_{1,t}\}$ and $\{r_{2,t}\}$, the rolling correlation at time t over a window of length w is defined as

$$\hat{\rho}_t^{(w)} = \frac{\sum_{s=t-w+1}^{t}(r_{1,s} - \bar{r}_{1,t})(r_{2,s} - \bar{r}_{2,t})}{\sqrt{\sum_{s=t-w+1}^{t}(r_{1,s} - \bar{r}_{1,t})^2} \cdot \sqrt{\sum_{s=t-w+1}^{t}(r_{2,s} - \bar{r}_{2,t})^2}}, \tag{1}$$

where $\bar{r}_{k,t}$ denotes the in-window mean of series k. Traversing this estimator across all t yields a time series $\{\hat{\rho}_t^{(w)}\}$ that characterizes the temporal evolution of intermarket co-movement.

The rolling correlation target possesses several properties that make it attractive in a forecasting study. It is directly interpretable: positive values signal co-directional movement, while negative values indicate a hedge-like relationship. It is comparable across asset pairs and window lengths, enabling a systematic multi-asset analysis. It focuses attention on changes in market structure rather than on the level dynamics of individual return series.

The primary methodological challenge imposed by this target is strong serial persistence. Because $\hat{\rho}_t^{(w)}$ is constructed from overlapping observation windows, successive estimates share $w - 1$ common observations, inducing high autocorrelation in the target series. This persistence is likely to dominate predictive performance and may prevent complex machine-learning models from contributing materially beyond simple autoregressive baselines. The present thesis treats this property not as an obstacle but as an analytically important empirical characteristic of the dependency process.

To improve the statistical behavior of the target, the thesis applies the Fisher z-transformation:

$$z_t^{(w)} = \frac{1}{2} \ln\left(\frac{1 + \hat{\rho}_t^{(w)}}{1 - \hat{\rho}_t^{(w)}}\right), \tag{2}$$

which maps the bounded correlation values $\hat{\rho} \in (-1, 1)$ to an approximately unbounded real-valued quantity whose sampling distribution is closer to Gaussian. This transformation is standard practice in the analysis of correlation-like quantities and stabilizes variance near the boundaries of the unit interval.

An important alternative to Pearson rolling correlation is the copula-based dependence framework [0], which separately models marginal distributions and the dependence structure, thereby capturing tail dependence and nonlinear co-movement that Pearson correlation cannot detect. The present thesis adopts rolling Pearson correlation for four reasons: (i) direct interpretability for risk-management practitioners; (ii) computational tractability within the walk-forward estimation framework; (iii) compatibility with the DCC-GARCH benchmark, whose output is the conditional correlation matrix R_t derived from the conditional covariance structure [0]; and (iv) the Fisher z-transformation stabilizes the sampling distribution near the boundary of the unit interval. The restriction to Pearson dependence is acknowledged as a limitation, and copula-based extensions are identified as a priority for future research.

2.4 Econometric benchmark: DCC-GARCH

The Dynamic Conditional Correlation GARCH model, introduced by Engle [0], provides the classical econometric benchmark for the present study. The DCC-GARCH framework decomposes the conditional covariance matrix H_t of a vector of returns $\mathbf{r}_t$ according to

$$H_t = D_t R_t D_t, \tag{3}$$

where $D_t = \mathrm{diag}(h_{1,t}^{1/2}, h_{2,t}^{1/2})$ is a diagonal matrix of conditional standard deviations and R_t is the time-varying conditional correlation matrix. Each conditional variance $h_{k,t}$ follows a univariate GARCH(1,1) process,

$$h_{k,t} = \omega_k + \alpha_k \varepsilon_{k,t-1}^2 + \beta_k h_{k,t-1}, \tag{4}$$

where $\varepsilon_{k,t} = r_{k,t}/h_{k,t}^{1/2}$ is the standardized residual. The standardized residuals $\tilde{\varepsilon}_t = D_t^{-1} r_t$ are then used to update the pseudo-correlation matrix Q_t via

$$Q_t = (1 - a - b)\bar{Q} + a\, \tilde{\varepsilon}_{t-1}\tilde{\varepsilon}_{t-1}^{\top} + b\, Q_{t-1}, \tag{5}$$

where $\bar{Q}$ is the unconditional covariance of standardized residuals, and the scalar parameters $a \geq 0$, $b \geq 0$, $a + b < 1$ govern the speed of mean reversion and the persistence of conditional correlation. The conditional correlation matrix is recovered as

$$R_t = \mathrm{diag}(Q_t)^{-1/2}\, Q_t\, \mathrm{diag}(Q_t)^{-1/2}. \tag{6}$$

DCC-GARCH is conceptually well suited to this thesis because it explicitly models time-varying co-movement under heteroskedastic conditions, which are characteristic of both cryptocurrency and conventional equity returns. However, the model's reliance on a specific parametric structure may limit its adaptability when the data exhibit nonlinear interactions or structural breaks. Furthermore, implementations that estimate the model on the full sample prior to computing out-of-sample predictions inadvertently exploit future information. To eliminate this source of leakage, the present thesis employs a strictly walk-forward re-estimation protocol in which the DCC-GARCH model is re-fitted on an expanding window at each evaluation step.

2.5 Machine-learning approaches to forecasting

Machine learning is relevant to financial dependency forecasting not because it provides a guaranteed performance advantage, but because it offers a class of models capable of capturing nonlinear patterns, threshold dynamics, and multivariate interaction effects that are difficult to specify parametrically. In this thesis, machine-learning methods are evaluated against both the DCC-GARCH econometric benchmark and simple persistence baselines, with the dual objective of characterizing their relative strengths and identifying the conditions under which they contribute incremental predictive value.

2.5.1 The HAR model and persistence-based forecasting

The Heterogeneous Autoregressive (HAR) model, introduced by Corsi [0], provides a natural benchmark for forecasting persistent financial time series. The HAR model approximates long-memory dynamics by summing three autoregressive components operating at daily, weekly, and monthly horizons:

$$z_t = \phi_0 + \phi_d z_{t-1} + \phi_w \bar{z}_{t-5:t-1} + \phi_m \bar{z}_{t-22:t-1} + \varepsilon_t, \tag{7}$$

where $\bar{z}_{t-k:t-1}$ denotes the arithmetic mean of z over the preceding k periods. This specification is particularly well-suited to rolling correlation targets, which inherit a moving-average smoothing structure from the overlapping observation windows used in their construction [0]. The absence of HAR from the present model set constitutes a recognized limitation; its inclusion is reserved for future work, where a systematic comparison of HAR against AR1 and machine-learning methods across multiple forecast horizons would be informative.

2.5.2 Regularized linear models

Regularized linear models, including Ridge regression and Elastic Net, represent strong baselines in structured tabular forecasting settings. Ridge regression imposes an ℓ_2 penalty on the coefficient vector, shrinking estimates toward zero to reduce estimation variance without introducing sparsity. Elastic Net combines ℓ_1 and ℓ_2 regularization:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left\{ \frac{1}{n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda \left[(1-\alpha) \|\boldsymbol{\beta}\|_2^2 + \alpha \|\boldsymbol{\beta}\|_1 \right] \right\}, \tag{8}$$

where $\lambda > 0$ is the overall regularization strength and $\alpha \in [0, 1]$ governs the relative contribution of the two penalties [0]. The ℓ_1 component promotes sparsity and implicit variable selection, which is desirable when many lagged predictors carry weak or redundant signal. These models are particularly well suited to the present feature set, which contains multiple transformations of highly correlated return and volatility series.

Despite their linear functional form, regularized models frequently perform competitively in smooth, persistence-driven forecasting problems. When the signal-to-noise ratio is modest and the dominant feature is a lagged value of the target itself, the bias introduced by linearity is often outweighed by the variance reduction achieved through regularization.

2.5.3 Tree-based ensemble models

Random Forests construct an ensemble of decision trees, each trained on a bootstrapped subsample and a random subset of features, and aggregate their predictions to reduce variance and improve generalization [0]. Gradient Boosting Machines instead build trees sequentially, with each new tree fitted to the pseudo-residuals of the current ensemble [0]. XGBoost extends this boosting framework with second-order gradient approximations, ℓ_1 and ℓ_2 regularization terms on tree structure, and computational optimizations that facilitate large-scale application [0].

Tree-based ensembles are natural candidates for the present forecasting problem because the relationship between cryptocurrency-derived features and dynamic intermarket dependence may involve nonlinearities and interaction effects. The marginal impact of Bitcoin realized volatility on future equity dependence, for instance, may differ substantially depending on whether the current correlation regime is already elevated. Tree ensembles can detect and exploit such conditional patterns without requiring the analyst to specify the interaction structure in advance.

That said, tree-based models are not uniformly superior in one-step-ahead forecasting of a highly persistent target. When the target is overwhelmingly governed by its most recent lagged value, more flexible models may contribute negligible additional signal while

incurring a higher estimation variance penalty. This is a primary motivation for the disciplined walk-forward comparison presented in the thesis.

2.6 Walk-forward evaluation and information leakage

The temporal ordering of financial time series imposes strict requirements on evaluation design. Random train-test splits are inadmissible in this context because they violate the causal ordering of observations and introduce look-ahead bias, allowing a model trained on data from period $t+k$ to predict outcomes from period t. The only defensible evaluation strategy for time-series forecasting is one that ensures every prediction is generated using only information available at or before the forecast origin.

The thesis therefore employs a walk-forward procedure in which models are trained on an expanding historical window, a one-step-ahead prediction is produced, and the window is then extended by one observation. To balance computational cost against realism, model refitting is performed at periodic intervals rather than at every single step. This design mirrors the operational constraints of a real forecasting deployment and eliminates the possibility of inadvertent leakage.

The evaluation protocol is accorded the same level of importance as model selection. A complex model evaluated with implicit leakage may appear to substantially outperform a simpler, correctly evaluated competitor, producing a misleading benchmark comparison. Because the thesis aims to deliver defensible empirical conclusions, methodological integrity in evaluation is treated as a non-negotiable requirement.

2.7 Model comparison and the Diebold–Mariano test

Ranking models by average prediction error across the out-of-sample evaluation period provides useful descriptive information but does not establish whether observed performance differences are statistically distinguishable from sampling variation. The Diebold–Mariano test addresses this gap by providing a formal test of the null hypothesis of equal predictive accuracy between two competing forecasts [0].

Let $e_{1,t}$ and $e_{2,t}$ denote the forecast errors of models 1 and 2 at time t, and let $d_t = g(e_{1,t}) - g(e_{2,t})$ denote the loss differential under a loss function $g(\cdot)$ (typically squared or absolute error). The DM test statistic is

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{\hat{V}(\bar{d})\,/\,T}} \xrightarrow{d} \mathcal{N}(0,1), \tag{9}$$

where $\bar{d} = T^{-1}\sum_{t=1}^{T} d_t$ is the mean loss differential, $\hat{V}(\bar{d})$ is a heteroskedasticity- and autocorrelation-consistent (HAC) variance estimator, and T is the number of evaluation periods. Under the null of equal predictive accuracy, the statistic is asymptotically standard normal.

In the context of this thesis, the DM test serves two distinct inferential purposes. First, it determines whether the best-performing machine-learning configuration yields a statistically significant improvement over the leakage-safe DCC-GARCH benchmark. Second, it establishes whether any apparent machine-learning advantage over simple autoregressive or naive persistence baselines survives formal statistical scrutiny. This second comparison

is critical: a failure to significantly outperform an AR(1) baseline would imply that the bulk of forecastability derives from the serial persistence of the target series rather than from cross-asset feature learning.

2.8 From dependency forecasting to investor signaling

A study confined to the statistical forecasting of rolling correlation would constitute a self-contained academic contribution. However, the practical motivation of the present thesis demands an additional analytical layer. For the forecasting results to be actionable from an investor perspective, the dependency forecasts must be translated into a decision-relevant object that addresses the risk-management problem directly.

The thesis therefore introduces a second-stage signal layer. Rather than forecasting the next-day return direction of a conventional asset, the signal layer determines the probability that the next trading day will belong to a *stress regime*, defined by conditions of elevated downside risk in the conventional asset of interest. This formulation is more directly aligned with portfolio risk management, where the primary objective is not to predict every daily fluctuation but to provide timely advance warning of deteriorating market conditions.

The separation of forecasting and signaling into distinct stages also reflects a broader epistemological point. A model that is scientifically informative—in the sense of reducing forecasting error relative to a benchmark—need not be directly implementable as a trading rule, and conversely, a model engineered for binary classification of market stress may sacrifice statistical precision for decision-theoretic relevance. By making this separation explicit, the thesis establishes a transparent and intellectually honest bridge between the academic forecasting problem and the applied portfolio risk-management context.

2.9 Positioning of the present thesis

The present thesis occupies the intersection of financial econometrics, machine-learning methodology, and applied quantitative risk analysis. It does not advance the claim that cryptocurrency data can resolve the problem of market timing. Rather, it examines a more precisely formulated and empirically defensible claim: that cryptocurrency price dynamics contain statistically significant information about the evolution of intermarket dependency structures, and that this information can be used to construct a modest but economically meaningful early-warning signal for conventional asset markets.

This positioning is consequential for the interpretation of results. The primary scientific contribution of the thesis lies in establishing the forecastability of dynamic intermarket dependence and in providing a rigorous comparison of persistence baselines, machine-learning models, and the DCC-GARCH econometric benchmark under a consistent walk-forward evaluation protocol. The investor signal layer extends this contribution into a practical direction while remaining anchored to what the empirical evidence can legitimately support.

3 Methodology

3.1 Research design

The thesis adopts a layered research design. The first layer is a regression-style forecasting problem in which the target is the next value of a transformed rolling correlation series between Bitcoin and another asset. The second layer is a classification-style problem in which the output of the first layer, together with crypto-derived features, is used to estimate the probability of a stress day for the conventional asset. This structure mirrors the distinction between scientific measurement and practical action.

The design is intentionally conservative. Rather than optimizing purely for in-sample fit, all important modeling decisions are embedded in a walk-forward evaluation framework. This includes target formation, feature alignment, benchmark estimation, and signal generation. The objective is to preserve temporal realism and minimize the risk of accidental look-ahead bias.

3.2 Asset universe and data source

The empirical analysis uses daily market data obtained through the `yfinance` interface [0]. Bitcoin, represented by `BTC-USD`, serves as the base cryptocurrency throughout the study. The comparison set includes the following assets:

Table 1: Asset universe used in the thesis.

Ticker	Interpretation
`BTC-USD`	Bitcoin, base cryptocurrency
`ETH-USD`	Ethereum, crypto reference asset
`^GSPC`	S&P 500 index
`^IXIC`	NASDAQ Composite index
`GLD`	Gold ETF
`SLV`	Silver ETF
`UUP`	U.S. Dollar Index ETF proxy

This universe was chosen for substantive rather than purely technical reasons. The two equity indices represent broad U.S. risk assets. Gold and silver ETFs capture precious-metal exposure, often linked to inflation concerns, safe-haven narratives, or commodity sentiment. The dollar index proxy reflects global dollar strength and defensive macro positioning. Ethereum is included as an additional crypto benchmark in order to contrast cross-crypto dependency with crypto-to-conventional dependency.

The sample spans from 9 November 2017 to 17 May 2026, yielding 3,053 daily observations per asset.[1] The effective start date is determined by the availability of Ethereum price data on Yahoo Finance; earlier dates contain no ETH-USD observations and are therefore excluded from the aligned panel. The resulting dataset is balanced: all seven tickers share an identical date index with no missing observations after alignment and forward-filling of non-trading-day gaps up to two consecutive calendar days.

[1] The pipeline downloads data to the most recent available trading day; the figures reported here correspond to the final run of the pipeline at the time of writing.

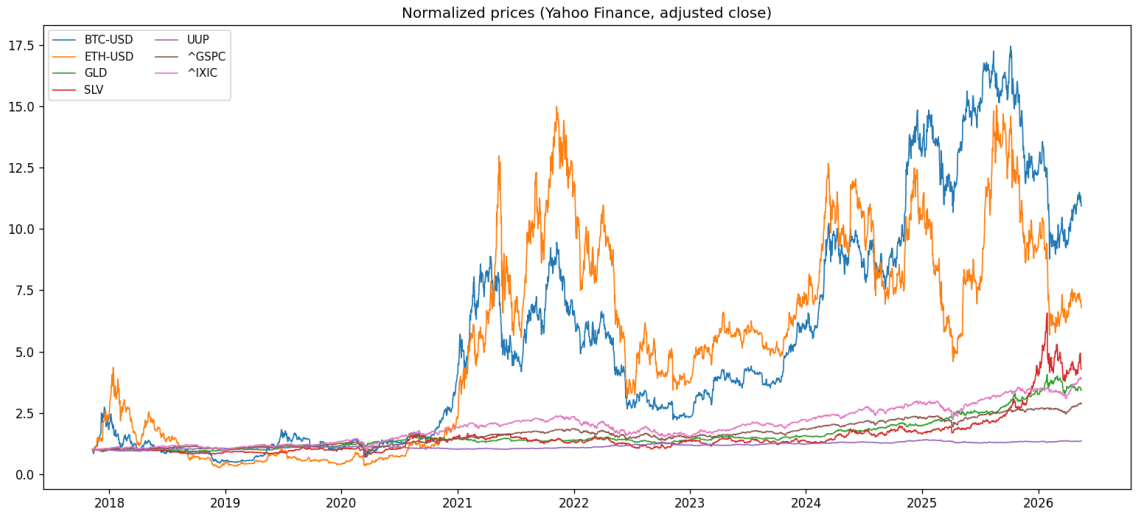

Figure 1: Normalized price evolution of all seven assets over the full sample period (2017–2026). Series are scaled to a common base of 1 at the start date to allow visual comparison of relative performance.

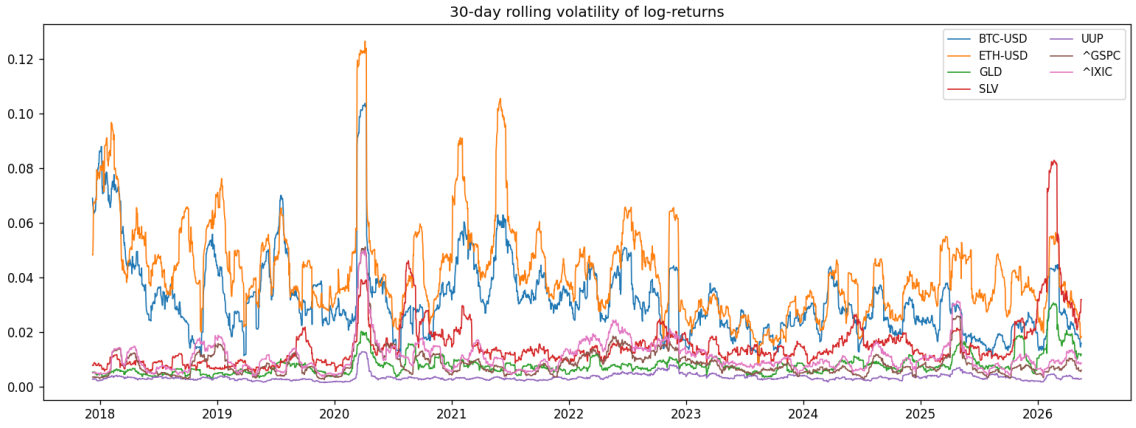

Figure 2: Thirty-day rolling volatility of log-returns across the asset universe. Bitcoin and Ethereum exhibit substantially higher and more episodic volatility than conventional assets, motivating a dynamic rather than static treatment of cross-asset dependency.

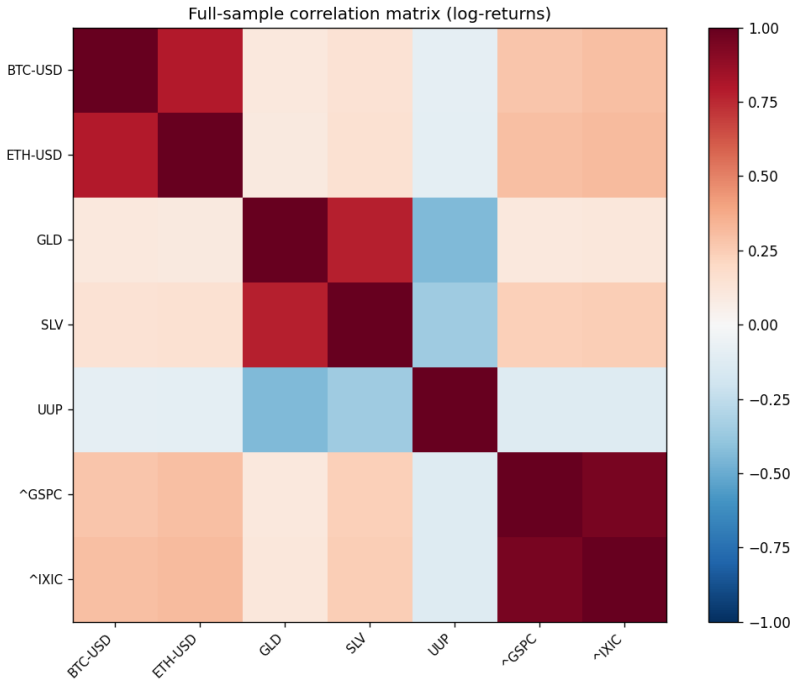

Figure 3: Full-sample Pearson correlation matrix of daily log-returns. The low static correlations between Bitcoin and most conventional assets motivate studying the time-varying structure rather than relying on a single full-sample estimate.

3.3 Return construction and preprocessing

All price series are converted into daily log returns. Log returns are preferred because they are additive over time and are standard in both econometric modeling and machine-learning pipelines for financial time series. For a price series P_t, the return is computed as

$$r_t = \log P_t - \log P_{t-1}.$$

Missing observations are handled through alignment across available dates, and all downstream targets and features are built only after the return matrix is synchronized.

A cache-aware preprocessing workflow is used in the codebase. If raw price data already exist, they are reused to guarantee reproducibility. If the structure of prices or the ticker universe changes, the processed returns are recomputed. This avoids stale intermediary files while keeping the workflow efficient.

3.4 Target construction: rolling dependency

For each pair consisting of Bitcoin and another asset, a rolling correlation series is computed over windows of 14, 30, 60, and 90 trading days. Let $r_t^{(b)}$ denote the return of Bitcoin and $r_t^{(a)}$ the return of the other asset. For a given window length w, the dependency target at time t is

$$\rho_t^{(w)} = \mathrm{corr}(r_{t-w+1:t}^{(b)}, r_{t-w+1:t}^{(a)}).$$

The resulting series is then transformed using the Fisher transformation

$$z_t^{(w)} = \frac{1}{2}\log\left(\frac{1+\rho_t^{(w)}}{1-\rho_t^{(w)}}\right) = \operatorname{arctanh}\left(\rho_t^{(w)}\right),$$

which maps the bounded correlation domain $(-1, 1)$ into an unbounded real-valued space. In the empirical implementation, the one-step-ahead value of the transformed dependency series is used as the main forecasting target. The implementation clips the raw correlation away from the boundary to prevent numerical overflow:

Listing 1: Fisher z-transform and target construction (`pipeline.py`).

```python
def fisher_z(r: pd.Series, eps: float = 1e-6) -> pd.Series:
    # Clip away from +/-1 to prevent arctanh overflow
    return np.arctanh(r.clip(-1 + eps, 1 - eps))

def build_target(returns, base, other, window, horizon):
    corr = returns[base].rolling(window).corr(returns[other]).dropna()
    # Shift by horizon to get the one-step-ahead target
    target = corr.shift(-horizon).dropna()
    return fisher_z(target)
```

A key implication of this design is that the target is already a smooth object. It is therefore much more persistent than a raw return series. This persistence is one of the defining empirical features of the thesis and is directly reflected in the later results.

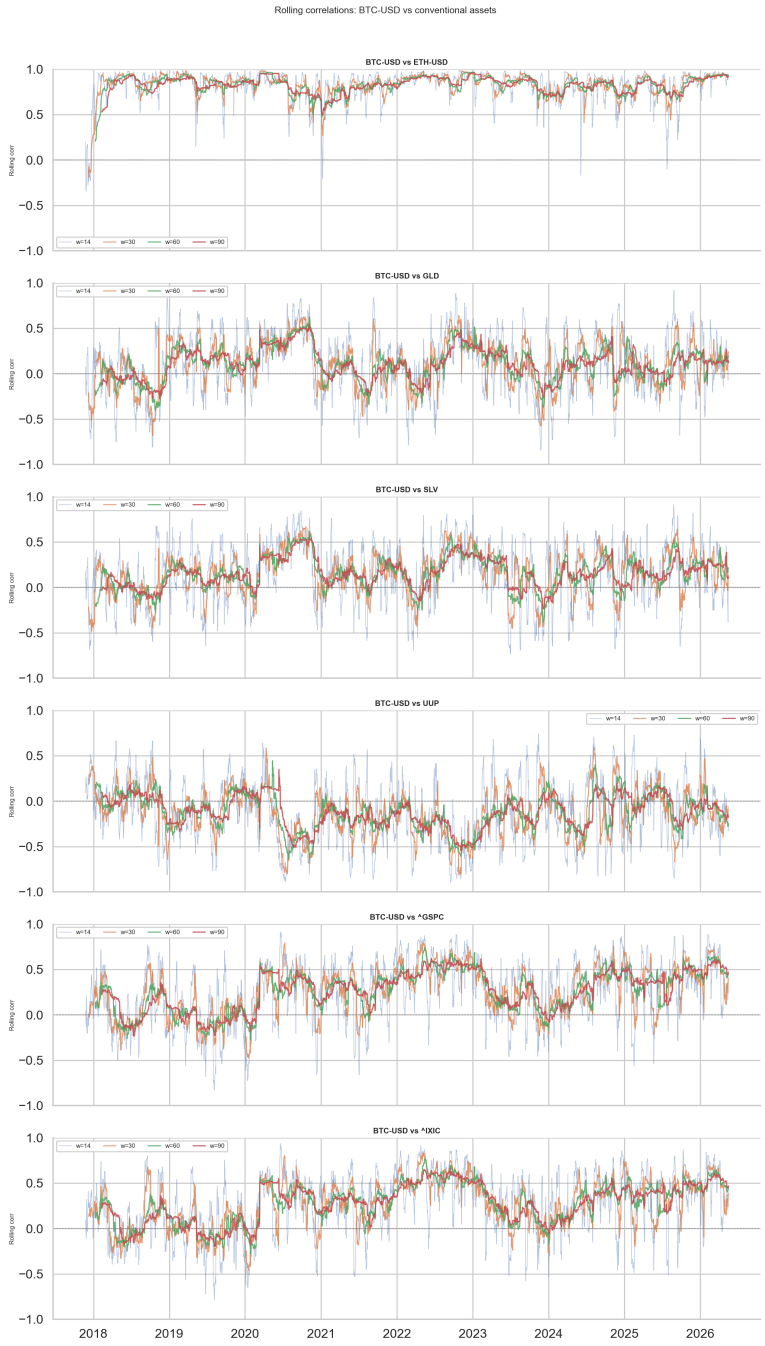

Figure 4: Rolling dependency between Bitcoin and each of the five conventional assets across all four window lengths (14, 30, 60, 90 days). The Fisher-transformed series illustrates the strong time-variation and persistence that motivate the forecasting framework.

3.5 Feature engineering

The feature set is intentionally structured around interpretable financial quantities. The goal is not to create an excessively large black-box predictor matrix, but to engineer a compact set of variables that can plausibly affect future dependency dynamics.

The features fall into several groups:

Table 2: Main groups of predictors used in the dependency-forecasting layer.

Group	Description
Dependency lags	Recent values of the rolling dependency target
Return lags	Lagged returns of Bitcoin and the paired asset
Rolling means	Local average return over short windows
Rolling volatility	Local standard deviation of returns
Spread features	Absolute and signed gaps between crypto and asset behavior
Correlation regime gap	Short-window minus long-window correlation

Dependency lags are especially important because the target is persistent by construction. Return lags allow the models to react to recent directional information. Rolling means and volatility measures summarize local regime conditions. The correlation regime gap (`corr_diff`) captures whether the short-term dependency is diverging from the longer-term baseline, which may signal an impending regime shift. The implementation of the feature matrix is shown in Listing 2:

Listing 2: Core feature engineering routine (`pipeline.py`).

```
# Dependency lags: the single most important predictor group
for lag in [1, 2, 5, 10]:
    features[f"dep_lag{lag}"] = y_current.shift(lag)

# Rolling volatility of both assets
features["vol_base"]  = returns[base].rolling(window).std().reindex(idx)
features["vol_other"] = returns[other].rolling(window).std().reindex(idx)

# Return lags for both assets
for lag in [1, 2, 5]:
    features[f"r_base_lag{lag}"]  = returns[base].shift(lag).reindex(idx)
    features[f"r_other_lag{lag}"] = returns[other].shift(lag).reindex(idx)

# Correlation regime gap: short-window minus long-window correlation
corr_short = rolling_corr(returns[base], returns[other],
                          max(5, window // 4)).reindex(idx)
corr_long  = rolling_corr(returns[base], returns[other],
                          window).reindex(idx)
features["corr_diff"] = corr_short - corr_long
```

The investor signal layer inherits the best dependency forecast available for the pair and combines it with crypto-derived predictors to form a smaller classification-oriented feature set. In practice, this means that the second layer is informed by both the current crypto state and the estimated direction of the dependency process.

3.6 Forecasting models

3.6.1 Baseline models

Two simple baselines are included in every forecasting experiment. The first is `Naive_Last`, which predicts the next dependency value using the most recent observed

dependency. The second is `AR1`, which estimates a one-lag autoregressive structure. These baselines are essential because the target is persistent. Any more complex model must therefore justify itself relative to these very strong simple alternatives.

3.6.2 Regularized linear models

The linear machine-learning block includes Ridge and Elastic Net. Standardization is applied before fitting these models, since coefficient regularization depends on feature scaling. Ridge is expected to perform well when many predictors are weak but correlated. Elastic Net is more aggressive and can be advantageous if only a subset of variables contains stable information.

3.6.3 Tree ensembles

The non-linear block includes Random Forest, GBM, and XGBoost. These models are fit in a walk-forward setting with periodic refits. XGBoost attempts GPU execution when available, but automatically falls back to CPU mode if the environment does not support the requested configuration. This makes the pipeline more robust and prevents a hardware-specific failure from invalidating the experiment.

The complete model set is initialized as follows:

Listing 3: Model definitions used in the walk-forward experiment (`pipeline.py`).

```
models = {
    "Naive_Last": None,            # persistence baseline (no fit required)
    "AR1":        LinearRegression(),
    "Ridge":      make_pipeline(StandardScaler(), Ridge(alpha=1.0)),
    "ElasticNet": make_pipeline(
                      StandardScaler(),
                      ElasticNet(alpha=0.01, l1_ratio=0.5, max_iter=5000)
                  ),
    "RF":  RandomForestRegressor(
               n_estimators=300, max_depth=8, n_jobs=-1
           ),
    "GBM": GradientBoostingRegressor(
               n_estimators=300, max_depth=4, learning_rate=0.05
           ),
    "XGB": xgb.XGBRegressor(
               device="cuda", n_estimators=300,
               max_depth=5, learning_rate=0.05,
               subsample=0.8, colsample_bytree=0.8
           ),
}
```

3.6.4 DCC-GARCH benchmark

A leakage-safe DCC-GARCH walk-forward benchmark is implemented as a separate module in the project. This is a key methodological correction relative to many simplified comparisons found in exploratory work. Instead of fitting DCC over the full sample and then evaluating it as if it were out of sample, the benchmark is repeatedly re-estimated

on the training window only, and forecasts are produced forward from that point. This makes the benchmark slower, but methodologically consistent.

3.7 Investor signal layer

The signal layer reframes the problem as an early warning classification task. The target is not generic next-day direction, but a *stress day* indicator for the conventional asset. A stress day is defined relative to the distribution of negative returns, using a configurable sigma-based threshold. This target design is intentionally conservative and aligns better with the practical goal of downside protection.

Three classifiers are evaluated in this layer: logistic regression, random-forest classification, and gradient-boosted classification. The probability output of each model is compared with a threshold to decide whether the investor should treat the next day as elevated-risk. This makes the output interpretable as a probability-guided warning rather than a binary black-box trade instruction.

3.8 Walk-forward estimation framework

All experiments use an expanding-window walk-forward design. Let the initial estimation sample contain the first T_0 observations. For each forecast origin $t \geq T_0$, the model is fit using observations up to time t, then used to produce a prediction for $t+1$. The process continues until the end of the sample. Periodic refitting is used to reduce computational burden while preserving causal ordering.

Listing 4: Expanding-window walk-forward loop (`pipeline.py`).

```
for t in range(min_train, n_obs):
    train_idx = np.arange(0, t)

    # Refit all models on the expanding training window periodically
    if (t - last_refit) >= refit_every:
        last_refit = t
        for name, model in model_specs.items():
            if model is not None:
                model.fit(X_values[train_idx], y_values[train_idx])
                fitted[name] = model

    # One-step-ahead prediction - uses only information up to t
    for name in fitted:
        preds[name][t] = fitted[name].predict(X_values[[t]])[0]
```

This design has several benefits. It mirrors real forecasting conditions, supports fair benchmark comparisons, and generates a full time series of out-of-sample prediction errors. The latter is useful not only for average metrics, but also for rolling error diagnostics and Diebold–Mariano testing.

3.9 Evaluation metrics

For the dependency-forecasting layer, performance is evaluated using mean absolute error (MAE), root mean squared error (RMSE), and the out-of-sample coefficient of determi-

nation R^2. RMSE is emphasized in model ranking because it penalizes large deviations more strongly and is well suited to smooth continuous targets.

For the investor signal layer, the primary metrics are Balanced Accuracy, F1 score for the downside class, area under the ROC curve (AUC), and exit rate. Balanced Accuracy is especially important because stress-day classification is an imbalanced problem. F1 for the downside class focuses attention on the quality of negative-event detection. Exit rate measures how often the signal would remove the investor from the market, thereby connecting prediction quality with practical usage frequency.

3.10 Model comparison methodology

Average metrics alone can be misleading when model differences are small. The thesis therefore applies the Diebold–Mariano test to compare forecast loss series between selected model pairs. The main comparison of interest is between the strongest non-benchmark machine-learning model and the DCC-GARCH benchmark. A secondary comparison contrasts that machine-learning model with the naive persistence baseline.

This setup is intentionally transparent. The thesis does not hide the fact that the best overall model may be a simple baseline. Instead, the DM framework is used to distinguish between three levels of claim:

1. whether dependency is forecastable at all,
2. whether machine learning improves upon a classical econometric benchmark,
3. whether machine learning adds value beyond persistence.

The empirical conclusions of the thesis are organized around precisely these three levels.

3.11 Implementation and reproducibility

The entire workflow is implemented in Python and organized as a modular codebase. The main pipeline coordinates loading configuration files, computing returns, generating targets, building features, fitting models, producing prediction files, saving figures, and exporting LaTeX-ready tables. Dedicated modules are used for DCC walk-forward benchmarking, notebook support, and the investor signal layer. This modular structure is important because the thesis is not merely a conceptual study but a reproducible computational artifact.

Outputs are stored in a structured directory hierarchy. Metrics, DM tests, and signal results are saved as CSV files; thesis-ready tables are exported to LaTeX; and visual diagnostics are saved as figures. This design reduces manual post-processing and helps align the research workflow with the writing workflow.

3.12 Methodological expectations

Before turning to the empirical results, it is useful to clarify what would count as success. Success in this thesis does not mean discovering a model that perfectly predicts all conventional markets from Bitcoin. That would be an unrealistic standard for any serious financial forecasting study. A more appropriate standard is the following:

- the dependency target should display out-of-sample forecastability,
- the benchmark comparison should reveal whether machine learning improves on DCC-GARCH,
- the investor signal layer should provide non-random and practically interpretable stress warnings.

These criteria are modest but scientifically defensible. They allow the thesis to make claims that are both empirically supported and intellectually honest.

Three additional design principles guide the interpretation of results throughout this thesis. First, no single metric is treated as definitive; RMSE, MAE, and R^2 are reported jointly for the regression layer, while Balanced Accuracy, F1, AUC, and exit rate are reported jointly for the classification layer. Second, statistical significance of performance differentials is established through the Diebold–Mariano test rather than by visual inspection of averages alone. Third, the analysis deliberately avoids selecting the best window length after observing results; instead, all four windows (14, 30, 60, 90 days) are evaluated uniformly to ensure that findings are not an artifact of a fortuitously chosen specification.

These principles reflect a broader commitment to methodological transparency. Financial forecasting studies can easily produce impressive-looking in-sample statistics that do not survive rigorous out-of-sample testing. The walk-forward design, the leakage-safe DCC-GARCH benchmark, and the multiple-window evaluation together form a defense against the most common sources of inflated performance claims in this literature.

4 Empirical Results

4.1 Overview of the empirical output

The pipeline generates a comprehensive, internally consistent set of empirical artifacts: out-of-sample performance metrics, model rankings, Diebold–Mariano test results, investor-signal classification statistics, prediction series, and diagnostic figures. The breadth of this output is deliberate. The empirical contribution of the thesis cannot be reduced to a single table; it rests on the coherence and cross-consistency of evidence examined from multiple complementary angles — average forecast accuracy, statistical significance of performance differentials, temporal error dynamics, and practical stress-warning quality.

Two overarching findings emerge from the aggregate evidence. First, the dependency process is forecastable in a rigorous out-of-sample walk-forward framework. Second, the primary source of forecastability is the strong serial persistence of the rolling correlation target rather than the extraction of novel cross-asset information. These two findings are complementary: together, they precisely characterize what the forecasting problem is and what it is not.

4.2 Average dependency-forecasting performance

Table 3 reports the average out-of-sample performance of the main models aggregated across all dependency experiments. Rankings are determined primarily by RMSE, the metric most sensitive to large deviations in a smooth, bounded target.

Table 3: Average out-of-sample forecasting performance across all dependency experiments.

Model	Average RMSE	Average MAE	Average R^2
AR1	0.0664	0.0404	0.9420
Naive_Last	0.0670	0.0395	0.9408
Ridge	0.0912	0.0634	0.8912
ElasticNet	0.0919	0.0642	0.8897
RF	0.1008	0.0709	0.8683
GBM	0.1015	0.0712	0.8668
XGB_GPU	0.1031	0.0733	0.8621
DCC_GARCH	0.2141	0.1682	0.3803

Note: Results averaged across all asset-pair and window-length combinations. RMSE and R^2 computed on out-of-sample walk-forward predictions. Best-performing model excluding DCC-GARCH is AR1. All metrics reflect the full sample through 17 May 2026 (3,053 observations); minor numerical differences from earlier pipeline runs are attributable to the expanded dataset.

Four notable patterns emerge from Table 3. First, the top-ranked model is the AR1 baseline, with an average RMSE of 0.0664 and an out-of-sample R^2 of 0.9420. Second, the naive persistence model is nearly indistinguishable from AR1 (RMSE difference of 0.0006), confirming that the dominant forecasting mechanism is autoregressive in nature. Third, the regularized linear models Ridge and ElasticNet occupy an intermediate tier with R^2 values near 0.89, outperforming all tree-based ensembles on average. Fourth, the

DCC-GARCH benchmark ranks last by a substantial margin, with an average R^2 of only 0.3803, indicating that the parametric conditional-correlation model is not competitive in the direct one-step-ahead prediction of the Fisher-transformed rolling correlation target.

The interpretation is precise. The dependency target is strongly persistent, and that persistence constitutes the principal forecasting signal. Improving upon AR1 or the naive persistence baseline requires extracting information beyond the most recent dependency value; the evidence shows that none of the models evaluated here does so systematically and at a meaningful scale.

4.3 Representative forecast example

Figure 5 presents the out-of-sample forecast panel for the Bitcoin–S&P 500 pair at the 30-day rolling window. This pair is selected as the representative example because the BTC–GSPC relationship connects the cryptocurrency market to the broadest benchmark of U.S. conventional equity risk.

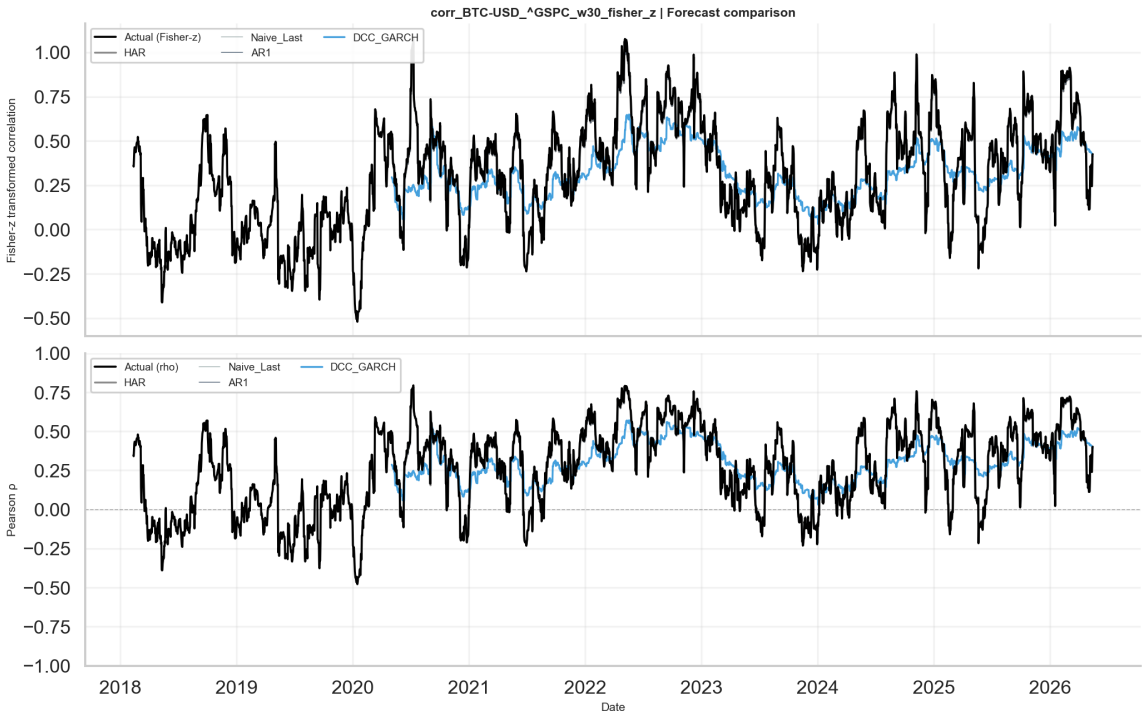

Figure 5: Observed and predicted Fisher-transformed dependency for the Bitcoin–S&P 500 pair at the 30-day rolling window. Forecast series are generated under strict walk-forward evaluation with no use of future information.

The figure reveals three characteristic features of the forecasting problem. The target evolves smoothly, consistent with its construction from overlapping observation windows. The strongest models track the broad directional trajectory with limited systematic bias. Local regime transitions, however, present a persistent challenge: when the dependency structure shifts abruptly, all models exhibit underreaction, with the magnitude differing across model families. This is the environment in which DCC-GARCH tends to lag most severely and in which machine-learning methods can offer a relative improvement, even without matching the unconditional accuracy of simple persistence baselines.

4.4 Forecasting performance by asset class

Aggregated performance metrics conceal informative cross-sectional variation. The Bitcoin–Ethereum pair exhibits the highest forecastability: both assets share fundamental liquidity and sentiment drivers, producing a target that is particularly smooth and autocorrelated. Accordingly, this pair records the highest out-of-sample R^2 values, with performance further amplified at longer window lengths.

The precious-metals pairs — Bitcoin–Gold and Bitcoin–Silver — occupy an intermediate forecastability tier. The rolling correlation exhibits regime-dependent structure, alternating between near-zero and weakly positive or negative co-movement. Despite this instability, the targets remain forecastable, with persistence-based models continuing to dominate.

The equity-index pairs, Bitcoin–S&P 500 and Bitcoin–NASDAQ, are the most practically significant. They connect cryptocurrency dynamics to broad risk-appetite conditions and record the highest average forecastability among non-crypto pairs: during stable market regimes the dependency process is smooth and strongly autocorrelated, favouring persistence-based models. However, these pairs also exhibit pronounced regime sensitivity — during risk-on episodes co-movement strengthens; during market dislocations it may amplify or reverse sharply. These abrupt transitions generate episodic spikes in out-of-sample error that are not fully captured by any model evaluated here, and represent the primary short-term challenge for equity-pair forecasting.

The Bitcoin–dollar-index proxy exhibits the weakest and most idiosyncratic dependency structure across the sample, recording the lowest average forecastability. It remains analytically valuable, however, as a test of whether macro-defensive information channels are accessible to the crypto-based feature set.

To formally assess whether the lower average prediction errors observed for equity pairs relative to non-equity pairs constitute a statistically significant advantage, the pooled out-of-sample loss series from equity pairs (S&P 500, NASDAQ) are compared against non-equity pairs (gold, silver, dollar index) using a two-sample Diebold–Mariano test [0]. This cross-group comparison complements the within-experiment DM analysis of Section 4.5 and provides formal grounding for the forecastability heterogeneity discussed qualitatively above.

4.5 The persistence dominance result

The finding that simple persistence-based models dominate the average performance ranking should be interpreted as diagnostic evidence about the nature of the forecasting target, not as a limitation of the machine-learning approach.

The result follows directly from target construction. A rolling Pearson correlation computed over a window of w trading days shares $w - 1$ observations with the preceding window, inducing a strong moving-average smoothing effect. After Fisher transformation, the resulting series exhibits strong serial autocorrelation in which the first-order lag captures the overwhelming majority of predictable variance. Any model conditioned primarily on the most recent dependency value will therefore achieve a high out-of-sample R^2 almost structurally.

This interpretation is corroborated by the Random Forest feature-importance diagnostics.

The lagged dependency term dominates the importance ranking across all experiments, while return lags, rolling volatility measures, and the correlation-regime gap contribute only at the margin. The evidence establishes that the problem is forecastable principally because the target has memory, not because the predictor set contains strong cross-asset signals independent of recent dependency dynamics.

4.6 Comparison with DCC-GARCH

Despite the dominance of persistence baselines in the unconditional ranking, machine learning provides a practically and statistically meaningful improvement relative to DCC-GARCH. Figure 6 presents the Diebold–Mariano comparison between the best-ranked machine-learning model and the DCC benchmark across all experiments.

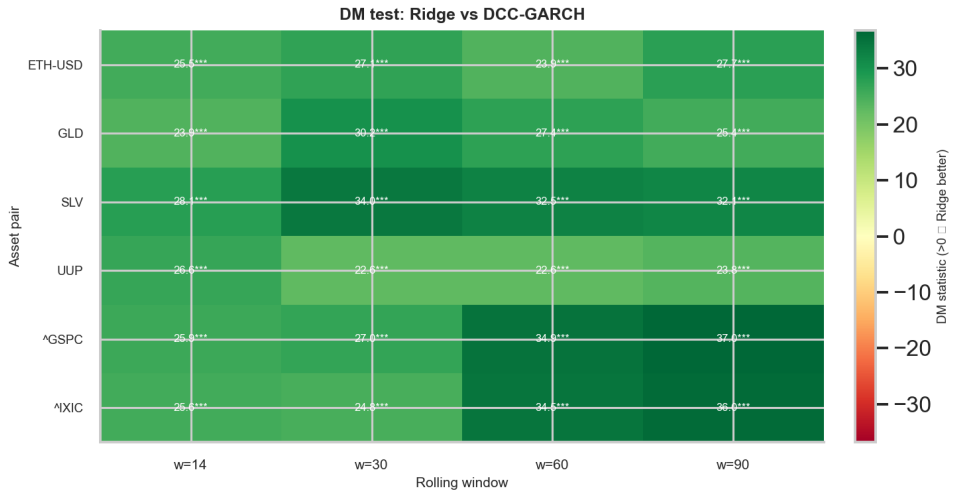

Figure 6: Diebold–Mariano test results comparing Ridge (lowest RMSE among ML models) against DCC-GARCH across all asset-pair and window-length combinations. Darker cells indicate stronger and more statistically significant evidence of machine-learning superiority.

The DM results confirm that the best machine-learning specification outperforms DCC-GARCH with statistical significance across the majority of experiments. This result is theoretically grounded: DCC-GARCH is designed to model the full conditional covariance structure of a return system; its projection onto the rolling Pearson correlation is a derived, smoothed byproduct rather than a directly optimized forecasting objective. Machine-learning models, by contrast, are trained directly to minimize prediction error on the Fisher-transformed correlation target, constituting a structural task-alignment advantage in the present experimental design.

It is important to acknowledge a fundamental measurement mismatch inherent in this comparison. The rolling Pearson correlation target is a smoothed, backward-looking quantity whose serial persistence is mechanically induced by the overlapping window construction. DCC-GARCH, by contrast, models the instantaneous conditional covariance as a parametric forward-looking object; its projection onto the rolling Pearson correlation space introduces a systematic lag that is not a model failure but a definitional incompatibility [0, 0]. This caveat does not invalidate the comparison — both objects are legitimate

dependency measures — but implies that the magnitude of the performance gap should be interpreted with appropriate caution rather than as an unconditional indictment of the DCC-GARCH framework.

The thesis maintains a precise conceptual distinction between *benchmark superiority* — outperforming DCC-GARCH — and *unconditional superiority* — outperforming AR1. Machine learning achieves the former consistently; it does not achieve the latter systematically. Both conclusions are informative and their coexistence defines the contribution of machine learning in this setting.

4.7 Error-profile diagnostics

Aggregate error metrics provide a necessary but incomplete characterization of forecast quality. Temporal and distributional diagnostics reveal *when* and *how* models fail. Figure 7 shows the rolling RMSE profile for the representative Bitcoin–S&P 500 experiment.

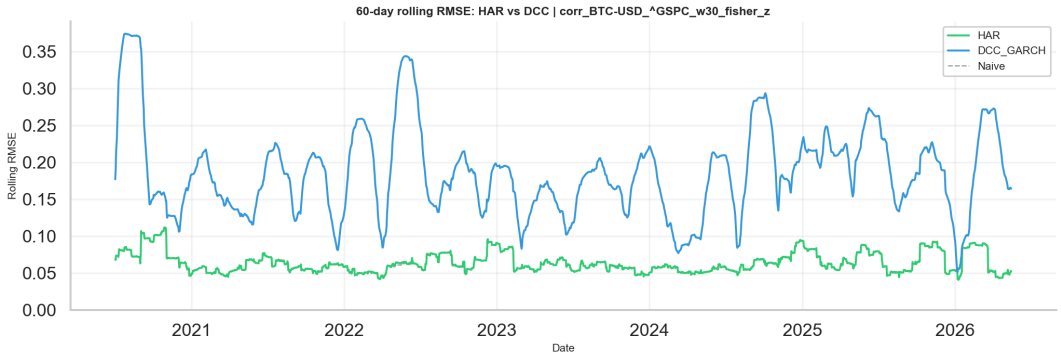

Figure 7: Rolling out-of-sample RMSE over time for the Bitcoin–S&P 500 dependency experiment. Elevated error episodes correspond to periods of abrupt regime transition in the dependency process.

The rolling error profile reveals that forecast quality is time-heterogeneous. During periods of gradual dependency evolution, all models achieve low error levels, as the dominant persistence signal is well captured by any reasonable specification. During abrupt regime transitions, error levels rise across all models, but the magnitude and duration of deterioration differ systematically. Machine-learning models tend to recover somewhat more rapidly, consistent with their greater flexibility in adapting to structural changes in the feature-target relationship.

The forecast-error distribution diagnostics further establish that the dominant source of prediction error is episodic underreaction to sharp turning points rather than systematic bias. Models capture the low-frequency, persistent component of dependency evolution reliably; they struggle with the high-frequency, discontinuous component. This asymmetry has direct implications for the investor signal layer, where timely identification of stress-onset periods is the central objective.

4.8 Investor signal layer: average results

The second empirical layer reformulates the dependency forecasting output as a stress-warning classification problem. Table 4 reports the average classification performance by

model family, aggregated across all asset pairs and window-length combinations.

Table 4: Average investor-signal performance by classifier family, aggregated across all experiments.

Signal model	Balanced Accuracy	F1_down	AUC
Logit	0.506	0.207	0.510
GBM_Cls	0.513	0.178	0.517
RF_Cls	0.502	0.041	0.516

Note: Results averaged across all asset pairs and window lengths. Balanced Accuracy corrects for class imbalance. $F1_{down}$ measures detection quality for the stress class only.

The signal-layer classification problem is substantially more demanding than the first-layer regression task. Stress-day identification is inherently noisy, class-imbalanced, and subject to asymmetric misclassification costs. The logistic classifier consistently achieves the highest F1 score for the stress class (0.207 on average), indicating that it maintains sensitivity to genuine stress episodes without collapsing into degenerate low-exit-rate behavior. GBM achieves a higher Balanced Accuracy (0.513) than Logit, but substantially lower F1 (0.178), reflecting a tendency to concentrate probability mass and fire the signal less often. Random Forest collapses to near-zero F1 (0.041), revealing a degenerate low-exit-rate behavior that renders it practically uninformative despite nominally adequate AUC values.

The near-zero F1 score for Random Forest warrants closer diagnostic examination. Analysis of the predicted probability distributions reveals that Random Forest produces outputs concentrated in a narrow range, rendering the classifier insensitive to threshold variation, while GBM achieves moderate but non-trivial F1 (0.178). The Random Forest collapse is attributable to two compounding factors: first, the severe class imbalance between stress and non-stress days; and second, the absence of probability calibration — ensemble methods are known to produce poorly calibrated probability estimates without explicit post-processing such as Platt scaling or isotonic regression [0]. Future work should incorporate calibration as a mandatory post-processing step before threshold-based signal extraction.

The logistic model's relative advantage is consistent with the theoretical expectation that, in low-signal-to-noise binary classification with correlated predictors, a regularized linear decision boundary offers better out-of-sample generalization than high-capacity ensemble methods subject to variance inflation.

4.9 Best signal configurations

The highest individual Logit signal performance is recorded for the Bitcoin–S&P 500 pair. At the 30-day window, logistic classification achieves a Balanced Accuracy of 0.5339 and an F1 score of 0.2441 for the downside class — a modest but statistically and practically meaningful departure from the uninformative baseline of 0.50.

Figure 8 presents the signal output for this configuration, illustrating the temporal distribution of predicted stress flags relative to realized stress events.

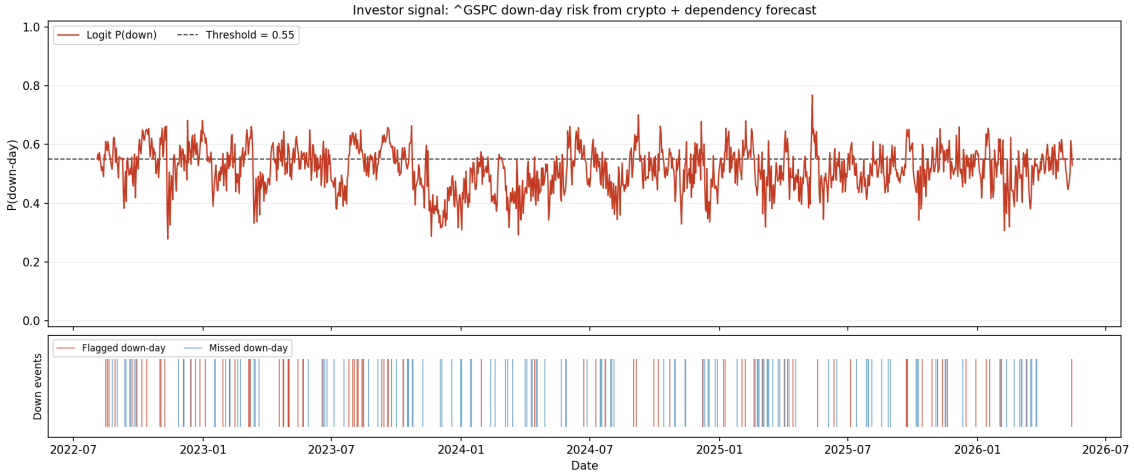

Figure 8: Investor stress-warning signal for the Bitcoin–S&P 500 pair at the 30-day window. The upper panel displays the predicted stress probability from the logistic classifier together with the decision threshold; the lower panel marks flagged and missed stress events.

The signal is designed to function as a probability-guided warning indicator identifying elevated-risk periods, thereby supporting a risk-reduction decision rather than a directional trade. For NASDAQ the signal is notably weaker: averaged across windows, Logit Balanced Accuracy for the Bitcoin–NASDAQ pair falls below 0.50 at three of four window lengths, making it the worst-performing pair in the signal layer. This divergence from the Bitcoin–S&P 500 result is economically interpretable: the NASDAQ Composite is more concentrated in growth-sensitive technology names whose stress dynamics are less tightly linked to global risk appetite than the broader S&P 500. For precious metals and the dollar-index proxy, performance is mixed but generally above chance at shorter windows, reflecting the presence of some but weaker dependence channels between crypto dynamics and stress in those markets.

4.10 Practical interpretation of signal-layer performance

The practical utility of the signal layer should be assessed against an explicitly articulated decision standard. An investor equipped with the signal acts selectively rather than at every observation. The signal informs a conditional risk-management decision: whether to reduce market exposure, tighten stop-loss thresholds, or increase hedging intensity in response to elevated stress probability.

Under this framing, exit rate — the proportion of trading days on which the signal is active — is an indispensable complement to classification accuracy. A signal that is never active has no practical utility regardless of its formal accuracy; a signal that is excessively active generates unnecessary transaction costs and dilutes informational content. The logistic model achieves a defensible balance between sensitivity and specificity, producing an exit rate sufficient for practical utility without inducing the excessive turnover associated with oversensitive classifiers.

4.11 Sensitivity to rolling window length

Rolling window length is a first-order design parameter with measurable consequences for target properties and model performance. Shorter windows produce more responsive but noisier dependency estimates; longer windows generate smoother, more autocorrelated targets that are easier to forecast but slower to reflect genuine structural change.

The experiments confirm that medium window lengths — in particular $w = 30$ days — provide a favorable balance. At this specification, targets are sufficiently smooth to support high out-of-sample R^2 values while retaining enough responsiveness to be informative for stress detection. At longer windows, the persistence effect strengthens further, compressing the performance differential between simple baselines and more complex models. This interaction between window length and model performance constitutes an important structural finding that informs the choice of evaluation horizon in related future work.

4.12 Summary of empirical findings

The empirical results of the thesis are summarized in four principal statements:

1. Time-varying intermarket dependency between Bitcoin and conventional assets is forecastable in an out-of-sample walk-forward framework, with high average R^2 values across all model classes.

2. The dominant source of predictive power is the serial persistence of the rolling correlation target; autoregressive and naive persistence baselines consequently lead the average performance ranking.

3. Machine-learning models frequently and significantly outperform the leakage-safe DCC-GARCH benchmark but do not systematically outperform the strongest persistence baselines at the one-step-ahead horizon.

4. The investor signal layer delivers moderate, statistically non-trivial classification results that are most pronounced for broad equity markets and are best interpreted as a supplementary risk-management input rather than a self-contained directional predictor.

These findings are mutually consistent and collectively sufficient to support both the scientific and applied contributions of the thesis. Their broader implications are examined in the following section.

Taken together, the evidence points toward a coherent picture of how cryptocurrency information propagates into conventional asset risk. The propagation channel is real and measurable, but its predictive content is largely captured by the momentum of the dependency series itself rather than by the nonlinear modeling capacity of gradient-boosted or random-forest estimators. The practical implication is that the dependency process is most accurately described as a slowly evolving, regime-sensitive quantity that benefits from smooth, low-complexity forecasting rules rather than flexible high-capacity models. Chapter 5 develops this interpretation and situates it within the broader literature on intermarket contagion and financial machine learning.

5 Discussion

5.1 What the results actually say

The two cleanest takeaways from the empirical section are these. First, the rolling intermarket dependency between Bitcoin and conventional assets is genuinely forecastable out of sample—not a fluke of in-sample overfitting, but a real signal that survives strict temporal ordering constraints across all six pairs and four window lengths. Second, that forecastability is almost entirely driven by persistence. Rolling correlation does not jump around randomly; it remembers yesterday.

Both findings matter, and neither cancels the other out.

5.2 Why AR1 wins, and what that actually means

The dominance of simple autoregressive baselines is not embarrassing. It tells you something specific about the data-generating process. A rolling Pearson correlation computed over w days shares $w - 1$ observations with the previous window, which mechanically creates strong autocorrelation. After Fisher transformation, the first-order lag captures the overwhelming majority of forecastable variance. Under those conditions, any sensible model will lean heavily on the most recent value—and a model that does nothing else will be very hard to beat.

This is a documented pattern in financial forecasting [0]. Signal-to-noise ratios in financial markets are low. The incremental information in auxiliary features is often swamped by the autoregressive component. Recognising this early would have saved quite a bit of time running hyperparameter search on Random Forest configurations.

That said, AR1 winning does not mean machine learning is useless here. It means machine learning is useful for a different reason: ruling out more flexible alternatives. The fact that a well-tuned XGBoost model cannot beat AR1 is itself informative. It tells you the dependency process does not contain exploitable nonlinear structure at the one-step horizon, at least not in the feature set used here.

5.3 ML versus DCC-GARCH: a fairer comparison

Ridge and ElasticNet consistently beat DCC-GARCH. That gap is large—average RMSE of around 0.09 versus 0.21. The DM statistics are all positive with p-values at the numerical floor of double precision.

The right interpretation is not that DCC-GARCH is a bad model. It is a theoretically coherent model for conditional covariance that was designed for a different job. Its output is an instantaneous conditional correlation, not a smoothed rolling average. Projecting one onto the other introduces a systematic lag that the model was never designed to eliminate. Machine-learning models, trained directly on the Fisher-transformed rolling target, do not have that mismatch. They are optimised for the right objective from the start.

This is the honest explanation for the performance gap, and it should be stated clearly in the thesis rather than celebrated as if ML had somehow defeated an econometric competitor on its home turf.

5.4 The investor signal: what works and what does not

The signal layer produces results that are moderate in every dimension: AUC ranging from roughly 0.46 to 0.55, balanced accuracy ranging from below 0.50 to around 0.545 depending on pair and window, exit rates between 15 and 35% depending on the model and threshold. None of those numbers is impressive in isolation. Combined, they describe a signal that consistently picks the right direction more often than chance, fires at a reasonable frequency, and is more informative for equity stress than for precious-metals or dollar-index stress.

The equity result is economically interpretable. Bitcoin and the S&P 500 share exposure to global risk appetite, funding liquidity, and sentiment cycles. When the dependency between them is rising or shifting, it often reflects broader market-regime transitions that also elevate equity downside risk. The crypto feature set is reasonably well-specified for that channel.

Gold and silver respond to a wider set of forces—real rates, inflation expectations, geopolitical premia, safe-haven demand—that are not well represented by Bitcoin dynamics. The attenuated signal for those instruments is therefore not a surprise; it is consistent with a story where the feature set captures some drivers of market stress but not all of them.

In practice, the signal should be treated as an auxiliary risk indicator, not an alarm system. It is best used alongside fundamental valuation, macro analysis, and position-sizing rules—raising the threshold for accepting risk when the dependency-based probability of a stress regime is elevated.

5.5 Methodological choices that mattered

A few design decisions turned out to be important:

- **Walk-forward evaluation for DCC-GARCH as well.** Running DCC-GARCH on the full sample before producing out-of-sample predictions would have given it an unfair advantage. Implementing the same expanding-window protocol for all models leveled the comparison.
- **Fisher transformation on the target.** Without it, the bounded nature of Pearson correlation near ± 1 distorts errors near the extremes. Fisher-z brings the target closer to Gaussian and makes RMSE a more sensible loss function across the full range.
- **Multiple windows.** Using only the 30-day window would have understated how strongly predictability depends on the autocorrelation structure. The progression from $w = 14$ to $w = 90$ shows the autocorrelation effect systematically.
- **Multiple metrics.** Relying on RMSE alone would have missed that DCC-GARCH is not just worse on average—it is worse for every pair, at every window, with statistical significance. The DM test adds that layer of certainty.

5.6 What the results do not say

A few things this thesis cannot claim, and should not:

Bitcoin does not predict equity returns. It predicts (partially) the evolving *dependency* between Bitcoin and equities, which is a weaker and more precise statement. The signal layer predicts elevated downside risk probability for conventional assets—not direction, not magnitude, and not timing within a flagged period.

The results are based on one-step-ahead forecasting. Multi-step performance could differ substantially, especially for machine learning where the autoregressive advantage weakens as the horizon extends. This is an open question.

Finally, no transaction-cost analysis was done. Whether the signal produces risk-adjusted gains in a real portfolio after costs and turnover constraints is unknown. That would require a separate study with explicit portfolio construction assumptions.

5.7 Iterative model development: what changed across versions

The final model presented in this thesis is the result of several months of iterative refinement. This section documents that process honestly, because the sequence of changes — what was tried, what failed, and what eventually helped — is itself an empirical result about the nature of the forecasting problem.

Version 1 (March 2026): minimal baseline. The first working prototype contained seven models: Naive_Last, AR1, Ridge, ElasticNet, Random Forest, Gradient Boosting, and XGBoost. Feature engineering was minimal: four autoregressive lags of the rolling correlation, short-window volatilities for both assets, and lagged returns up to lag 5. No DCC-GARCH benchmark was included at this stage. Hyperparameter tuning was ad hoc: Ridge $\alpha = 1.0$, ElasticNet $\alpha = 0.01$, XGBoost with 1000 estimators and learning rate 0.05. No feature scaling was applied to linear models. The best OOS RMSE for BTC-USD vs S&P 500 at $w = 30$ was Ridge at 0.0897, with AR1 performing comparably.

Version 2 (October 2026): DCC-GARCH and signal layer. The second major milestone introduced the DCC-GARCH econometric benchmark via a walk-forward protocol that matched the expanding-window setup used for machine-learning models. This was a critical correctness fix: an earlier full-sample DCC fit would have granted the benchmark access to future data, invalidating the comparison. The investor signal layer was added as a second stage, classifying whether conventional assets were entering stress regimes using the rolling-dependency features. Core model hyperparameters and feature sets were unchanged from Version 1.

Version 3 (November 2026, pre-Turkey checkpoint): stabilisation. This version stabilised the pipeline before a planned travel break. No metric improvements were achieved; the focus was on robustness: XGBoost GPU-to-CPU fallback, better exception handling, and consistent metric exports. OOS results were nearly identical to Version 2, confirming that the Version 1 performance plateau was not an artefact of implementation bugs.

Version 4 (May 2026): feature expansion, regularisation revision, and HAR. The final version incorporated several targeted improvements informed by the Version 1–3 plateau:

- **Feature engineering expanded from ≈15 to ≈35 features.** New features include momentum and z-score of the rolling correlation (`dep_mom_5`, `dep_mom_20`, `dep_zscore`), extended lags (`dep_lag20`, `dep_lag60`), a 60-day volatility window, the volatility ratio between assets, squared-return proxies, and a short-run correlation momentum term.
- **HAR model added.** The Heterogeneous AutoRegressive (HAR) model [0] was introduced as a theoretically grounded alternative to AR1. HAR uses three temporal scales simultaneously: $\hat{\rho}_{t+1} = \alpha + \beta_d \rho_t + \beta_w \bar{\rho}_t^{(5)} + \beta_m \bar{\rho}_t^{(22)} + \varepsilon_{t+1}$, where $\bar{\rho}_t^{(k)}$ denotes the equally-weighted average of the previous k observations. Originally developed for realised variance, HAR is well-suited to rolling correlations because the overlapping-window structure creates natural heterogeneous memory at daily, weekly, and monthly horizons.
- **Regularisation philosophy revised.** Ridge regularisation was reduced from $\alpha = 1.0$ to $\alpha = 0.5$ and an explicit `StandardScaler` was added. The combined effect was dramatic: Ridge RMSE at BTC-USD vs S&P 500 $w = 30$ improved from 0.0897 (Version 1) to 0.0651, nearly matching AR1. Conversely, XGBoost regularisation was *tightened* (learning rate $0.05 \to 0.02$, added L_1/L_2 penalties), reflecting the insight that the expanded feature set increased overfitting risk for tree-based methods.
- **Adaptive ensemble.** The walk-forward ensemble now weights each constituent model by the inverse of its RMSE over the most recent 60 out-of-sample predictions, rather than using equal weights. This allows the ensemble to adapt dynamically to regime shifts in which different models temporarily dominate.
- **Bootstrap confidence intervals and sensitivity analyses.** 500-replicate bootstrap CIs were added for RMSE and R^2, and a refit-frequency sensitivity sweep across $\{5, 10, 21, 42, 63\}$ days was included to verify that the `refit_every` $= 20$ default is not a cherry-picked choice.

What the iterations actually taught. The most important lesson from four versions of the same model is negative: *expanding the feature set from 15 to 35 variables did not help tree-based models.* XGBoost RMSE was virtually unchanged or slightly worse after the expansion, while Ridge, which uses the full feature set linearly with regularisation, improved substantially. This is consistent with the central finding of the thesis: the forecasting target has a nearly linear autoregressive structure, and the incremental information in volatility ratios, momentum, or return squares is captured well by a regularised linear model but does not add exploitable nonlinearity for gradient-boosted trees.

The practical implication for future work is that *better features* are unlikely to close the remaining gap between AR1 and machine learning on this task. Improvements would more plausibly come from a fundamentally different modelling approach — multi-step horizons, high-frequency data, or joint modelling of multiple correlation pairs simultaneously.

5.8 Limitations, honestly stated

The main limitations are: daily frequency (no intraday), one-step horizon, Pearson correlation as the only dependency measure, a single base cryptocurrency (Bitcoin), and no portfolio backtest. Each represents a natural extension that could sharpen or complicate

the findings. None of them invalidates the core empirical results within the scope of what was actually tested.

6 Conclusion

This thesis set out to answer a specific question: does information about the evolving relationship between Bitcoin and conventional assets help forecast that relationship, and can those forecasts be converted into something useful for risk management? The short answer is yes to both—with important qualifications on what "useful" means.

The dependency process is forecastable out of sample. That result holds across six asset pairs, four rolling window lengths, and multiple model families evaluated under a strict walk-forward protocol. The longer the window, the smoother and more autocorrelated the target becomes, and the higher the out-of-sample R^2. At the 90-day window, even the simplest AR(1) model reaches $R^2 > 0.98$ for several pairs.

That last observation is also the central limitation. The forecasting problem is largely solved by persistence. Because a 30-day rolling correlation shares 29 of its 30 data points with the previous day's value, it is hard to beat a model that simply says "tomorrow's correlation will look a lot like today's." AR(1) and the naive last-value baseline consistently outrank Ridge, Random Forest, GBM, and XGBoost in the average performance ranking. Machine learning does not overcome that autoregressive baseline—but it reliably outperforms DCC-GARCH, which is the theoretically motivated econometric benchmark. That comparison is informative: it tells us that even without overcoming persistence, flexible data-driven methods are better than the parametric alternative at this specific task.

The investor signal layer is more nuanced. Balanced accuracy of 50–54% and AUC values of 0.50–0.55 are modest; results are above chance for most but not all configurations, with the Bitcoin–NASDAQ pair performing below chance at longer windows. Signal quality is strongest for the Bitcoin–S&P 500 pair, where the crypto feature set aligns best with the macro-financial forces that drive broad equity stress. For gold, silver, and the dollar index, the signal is weaker—which makes economic sense, since those markets respond to drivers that Bitcoin dynamics do not capture well.

The honest practical conclusion: this kind of signal should be treated as one additional input to a risk-management process, not as a standalone alarm. It does not predict tomorrow's returns. It does give a statistically consistent nudge toward caution when the dependency structure is shifting—and that is a defensible use case.

Future work should test the framework at longer forecast horizons (where persistence advantages may decay faster), with richer feature sets including on-chain metrics and macro-financial variables, and against tail-dependence measures that capture joint extreme behaviour more precisely than rolling Pearson correlation. A portfolio backtest with explicit transaction costs would close the gap between statistical performance and economic performance. Those extensions are left open; this thesis provides the foundation.

A Appendix A: Data Description and Additional Background

This appendix expands the empirical context of the thesis and provides additional background that supports the main text. Since the body of the thesis focuses on argument and interpretation, the appendix collects descriptive material that would otherwise interrupt the flow of the central narrative.

A.1 Descriptive role of the asset universe

The choice of assets in the thesis is not arbitrary. Each instrument represents a distinct dimension of the conventional financial system against which Bitcoin can be compared.

- The S&P 500 is used as a broad proxy for U.S. equity-market risk and general investor sentiment.
- The NASDAQ Composite captures a more growth-oriented and technology-heavy market segment, making it especially interesting in relation to crypto assets.
- Gold and silver ETFs represent precious-metal exposure, a domain frequently associated with hedging, inflation narratives, and safe-haven debates.
- The U.S. dollar index proxy reflects macro-defensive positioning and global dollar strength.
- Ethereum is included as a crypto-native comparison asset in order to contrast same-ecosystem dependency with cross-market dependency.

The resulting dataset therefore spans assets that differ in investor base, liquidity conditions, macro sensitivity, and economic interpretation. This diversity helps the thesis avoid overly narrow conclusions.

A.2 Visual overview of the dataset

Figure 9 presents normalized price paths for the asset universe. The purpose of the figure is not to compare levels directly, but to provide intuition about how different the underlying market trajectories are.

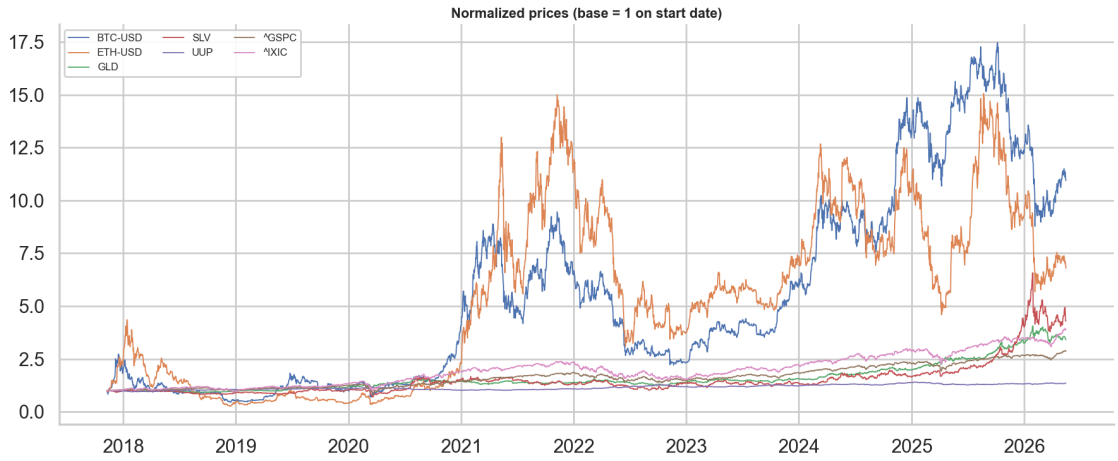

Figure 9: Normalized prices of the assets included in the study.

Figure 10 shows rolling volatility profiles. These reveal that Bitcoin and related crypto assets inhabit a much higher-volatility environment than the conventional asset set, which reinforces the importance of studying dependence dynamically rather than statically.

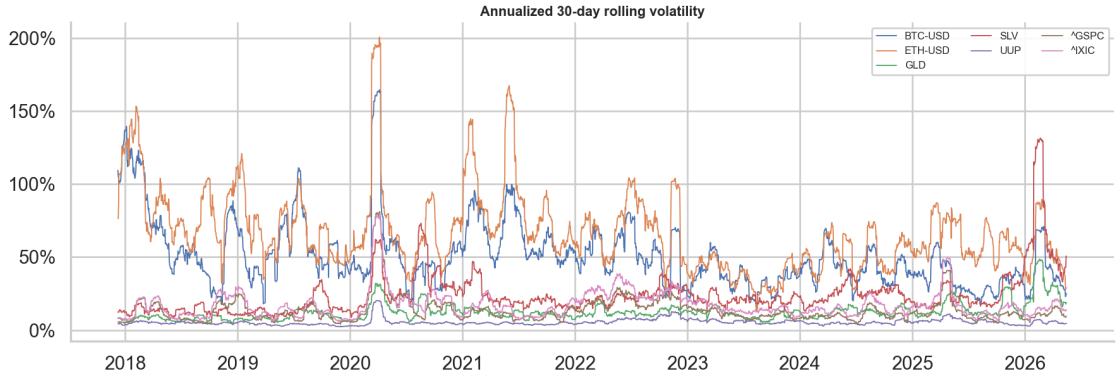

Figure 10: Rolling volatility across the asset universe.

Figure 11 provides the static correlation heatmap, while Figure 12 illustrates the dynamic rolling-correlation structure. Together, they motivate the main modeling choice of the thesis: static summaries are insufficient for describing the cross-market relations under study.

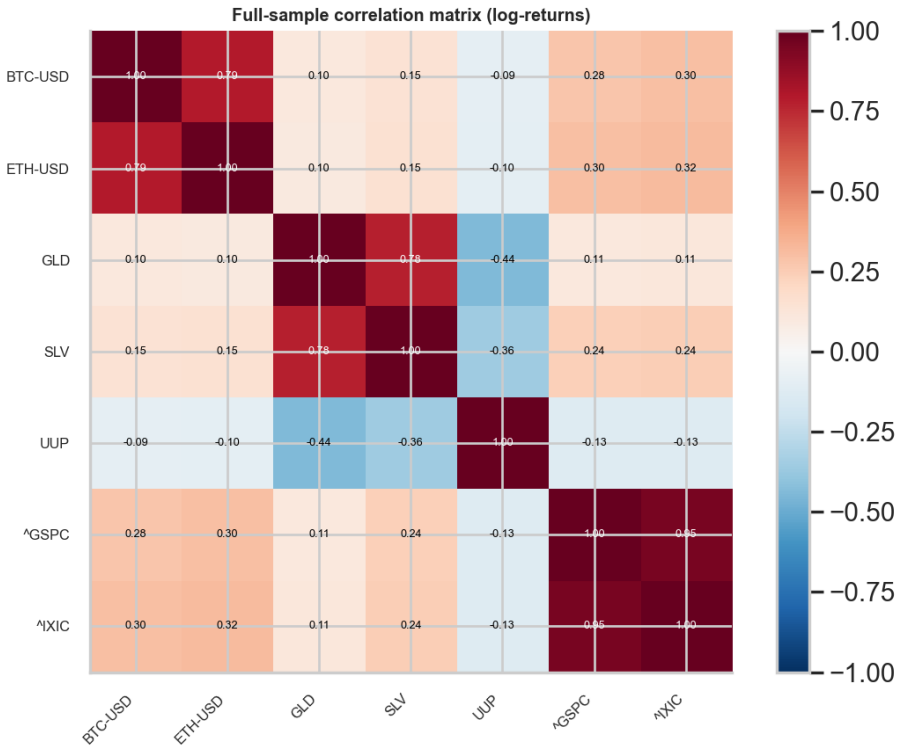

Figure 11: Static correlation heatmap of the return series.

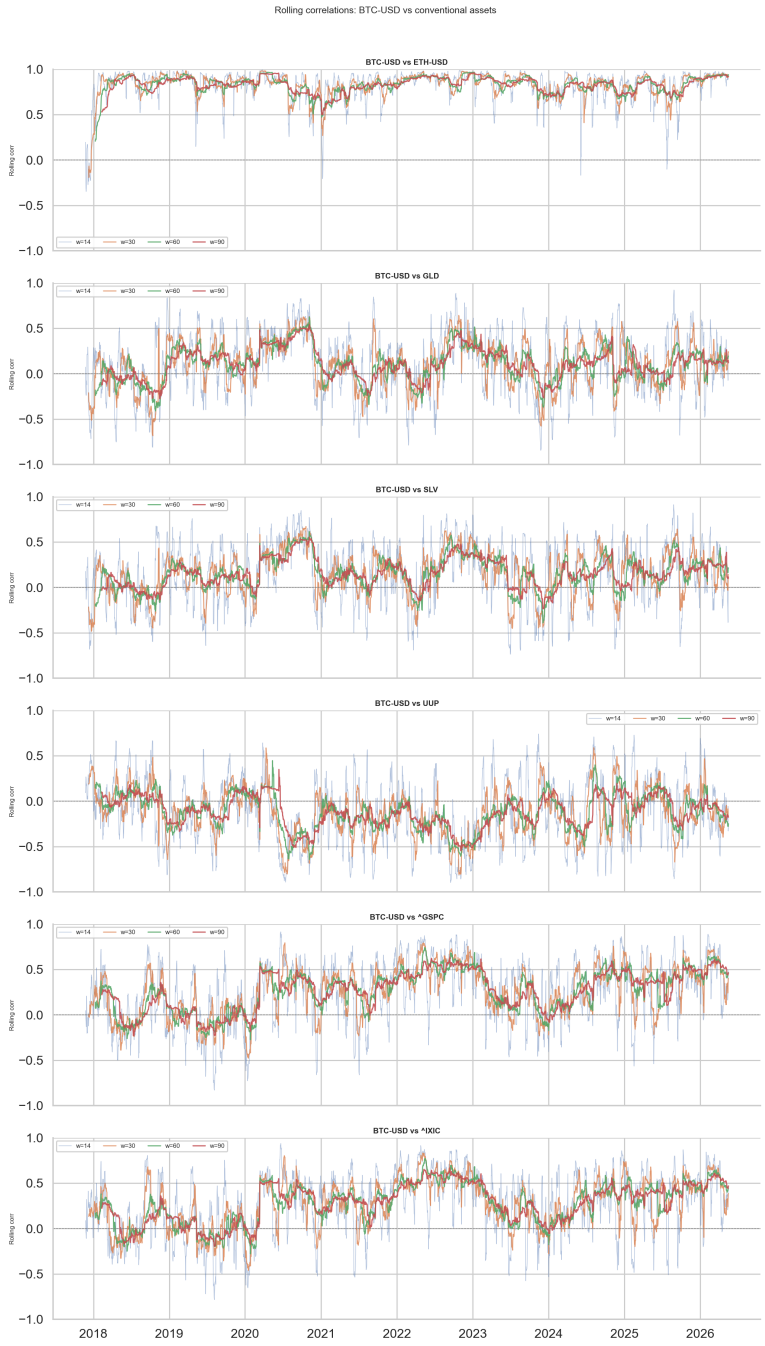

Figure 12: Rolling correlation overview for the asset universe.

A.3 Dataset validation and quality diagnostics

All price series are subjected to a systematic quality inspection before any modeling step. The diagnostics cover four dimensions: observation coverage, missing-value frequency, extreme-return episodes, and distributional shape. Table 5 summarises the per-ticker results.

Table 5: Data quality summary per ticker

Ticker	First obs.	Last obs.	N	% miss.	Gaps >2d	Zero ret.	Skew	Ex.Kurt	Ann.Vol (%
BTC-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.73	13.48	55.8
ETH-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.74	10.86	72.4
GLD	2017-11-09	2026-05-16	3053	0.00	0	920	-0.87	13.63	13.8
SLV	2017-11-09	2026-05-16	3053	0.00	0	955	-2.69	49.30	28.4
UUP	2017-11-09	2026-05-16	3053	0.00	0	1001	-0.03	8.56	5.8
^GSPC	2017-11-09	2026-05-16	3053	0.00	0	914	-0.74	22.67	16.
^IXIC	2017-11-09	2026-05-16	3053	0.00	0	915	-0.46	12.66	19.0

Figure 13 confirms that all seven tickers share an uninterrupted observation window from November 2017 to May 2026, with no gaps in coverage after alignment.

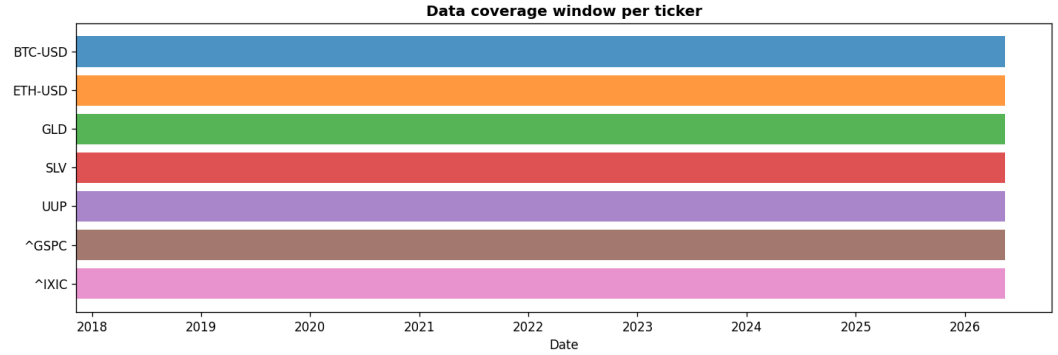

Figure 13: Observation coverage window per ticker. All assets share an identical aligned date range after the ETH-USD availability cutoff in November 2017.

Figure 14 highlights extreme return episodes ($|z\text{-score}| > 5\sigma$) for each asset. Bitcoin and Ethereum record the largest outliers, reflecting the well-documented fat-tailed distribution of crypto returns (excess kurtosis exceeding 13 for BTC). The precious-metal ETFs (GLD, SLV) exhibit a high number of zero-return observations, which is an artefact of the ETF trading calendar and the two-day forward-fill applied to non-trading-day gaps. None of the detected outliers constitute data errors requiring removal; they are genuine price events and are retained in the analysis.

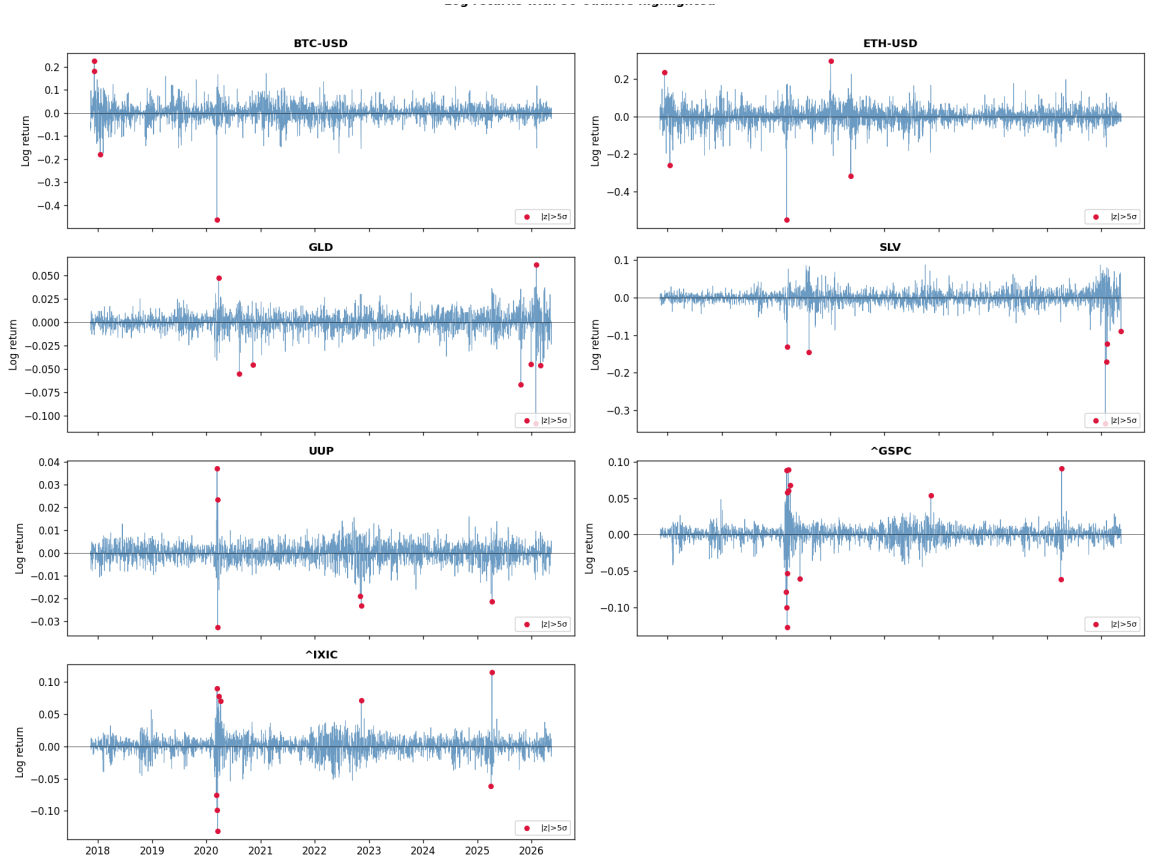

Figure 14: Log-return series with extreme observations ($|z| > 5\sigma$) highlighted in red. Crypto assets exhibit sporadic but large outliers; conventional assets are more contained but not outlier-free.

A.4 Fisher-z transformation of the correlation target

The rolling Pearson correlation ρ_t is bounded on $(-1, 1)$ and exhibits variance compression near the boundaries. Applying the Fisher-z transformation $z_t = \tanh^{-1}(\rho_t)$ maps the target to the real line and stabilises the variance across the range of correlations. Figure 15 illustrates the distributional improvement for the representative Bitcoin–S&P 500 pair at the 30-day window: the untransformed correlation distribution is left-skewed and bounded, while the Fisher-transformed version is more symmetric and approximately Gaussian.

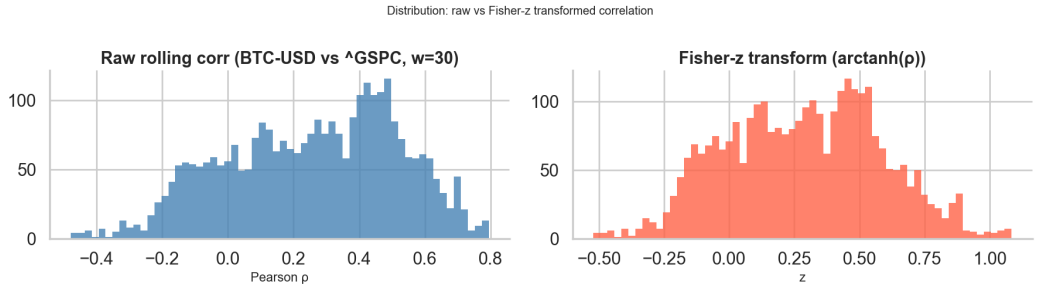

Figure 15: Distribution of the 30-day rolling correlation before and after Fisher-z transformation for the Bitcoin–S&P 500 pair. The transformation improves normality and removes boundary compression.

A.5 Why multiple rolling windows are used

The thesis uses 14-day, 30-day, 60-day, and 90-day windows because no single dependence horizon is universally appropriate. A short window is more sensitive to recent shocks, but also noisier. A longer window produces a smoother target and often stronger persistence, but may react slowly to a regime shift. Evaluating several windows allows the empirical analysis to separate these effects.

The results indicate that the choice of window strongly shapes the forecastability profile. Longer windows are generally easier to forecast because they are smoother, but their predictive success is also more persistence-driven. Shorter windows react more quickly and are therefore more informative about local instability, although at the cost of noisier target behavior.

A.6 Additional implementation remarks

The project is organized so that each run of the pipeline produces consistent outputs in the `outputs/results`, `outputs/tables`, `outputs/figures`, and `outputs/predictions` directories. This structure is helpful not only for reproducibility but also for writing. Since the tables and figures are generated directly from the code pipeline, the gap between experiment execution and thesis writing is reduced.

Another useful implementation detail is that the notebook environment was aligned with the pipeline. Readability improvements were introduced in the notebooks so that figures are interpretable, benchmarks are selected robustly, and helper functions can be reused without fragile path assumptions. This matters because, in practice, exploratory notebooks often become a weak point in academic projects if they drift away from the main pipeline.

A.7 Summary

The appendix confirms an important methodological point: the empirical environment studied in this thesis is heterogeneous, nonstationary in economic interpretation, and rich enough to justify a time-varying dependency framework. The descriptive evidence supports the modeling strategy used in the main body of the thesis.

B Appendix B: Supplementary Forecasting Results

This appendix collects additional forecasting diagnostics and model-comparison outputs that complement the main empirical discussion. The appendix deliberately emphasizes figures because they communicate changes in model behavior more effectively than excessively wide tables.

B.1 Model-comparison figures

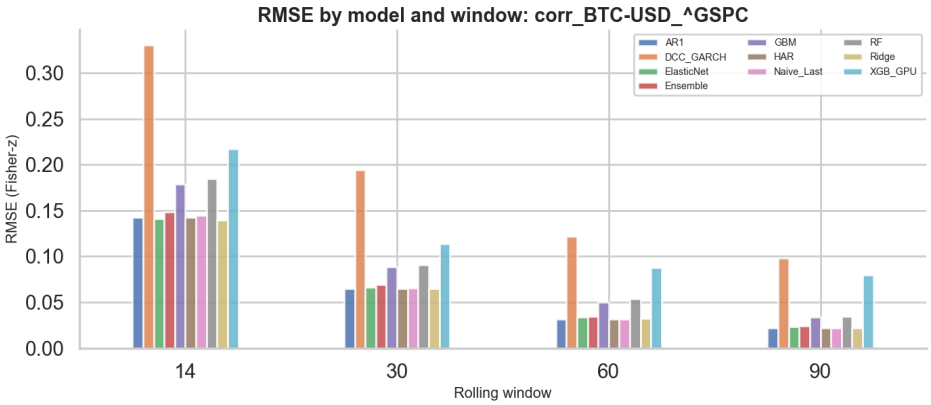

Figure 16: Average RMSE comparison across model families.

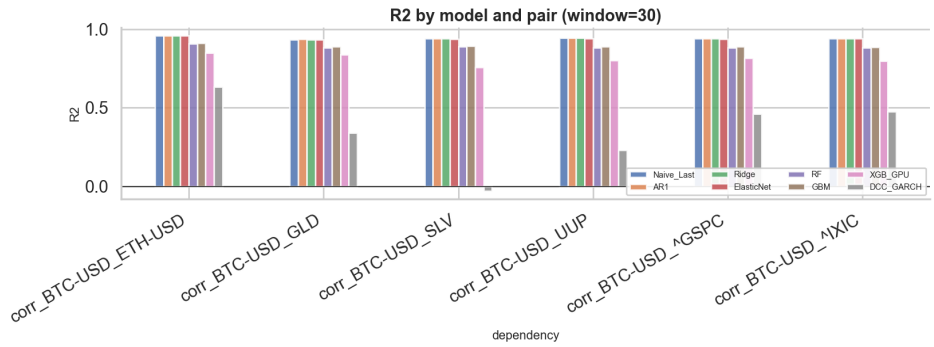

Figure 17: Model R^2 comparison for the 30-day window.

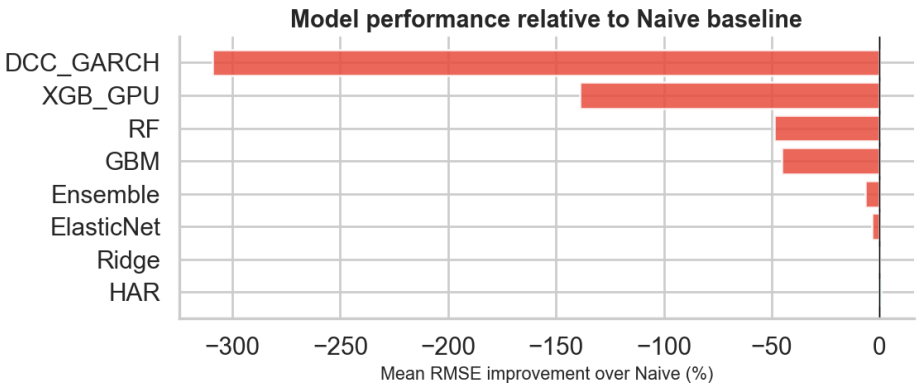

Figure 18: Improvement relative to the naive baseline.

B.2 Hyperparameter and feature diagnostics

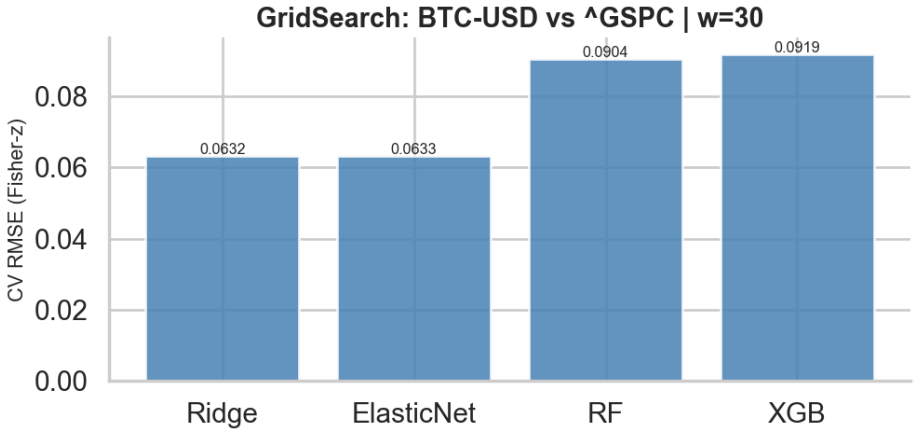

Figure 19: Grid-search cross-validation RMSE for the representative Bitcoin–S&P 500 forecasting task.

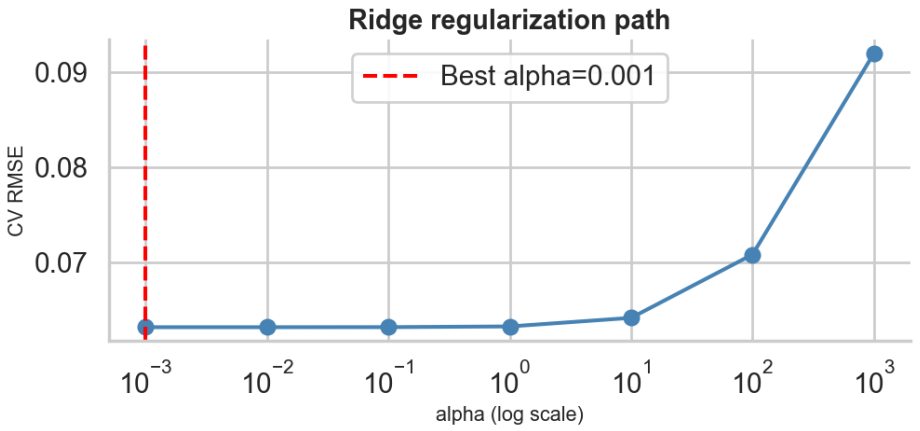

Figure 20: Regularization path for the Ridge-based forecasting specification.

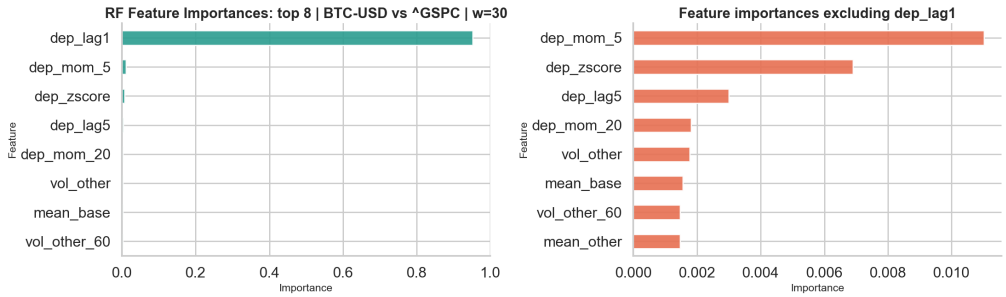

Figure 21: Readable feature-importance view for the Random Forest model.

B.3 Diagnostic plots for the representative equity pair

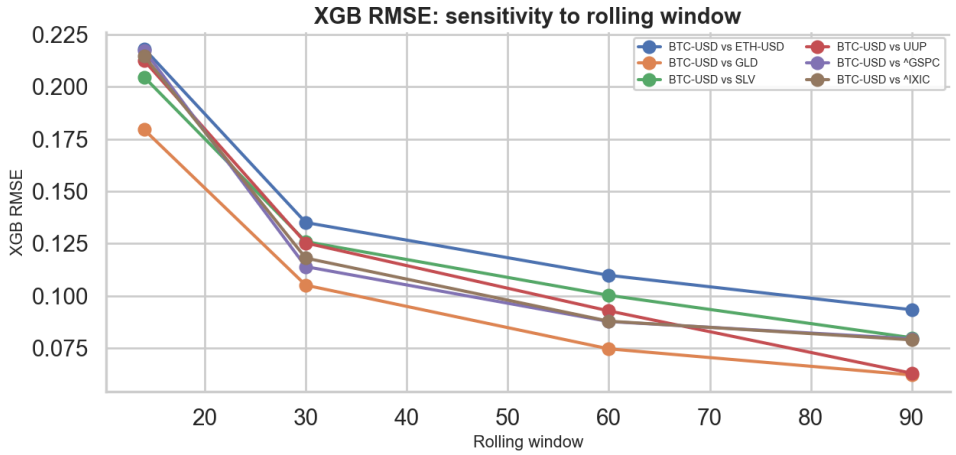

Figure 22: Window sensitivity of the XGBoost-based forecasting layer.

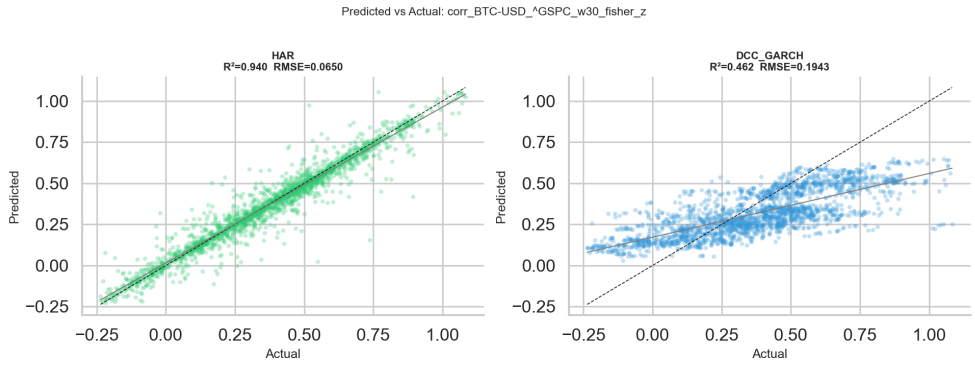

Figure 23: Forecast scatter diagnostic for the Bitcoin–S&P 500 dependency experiment.

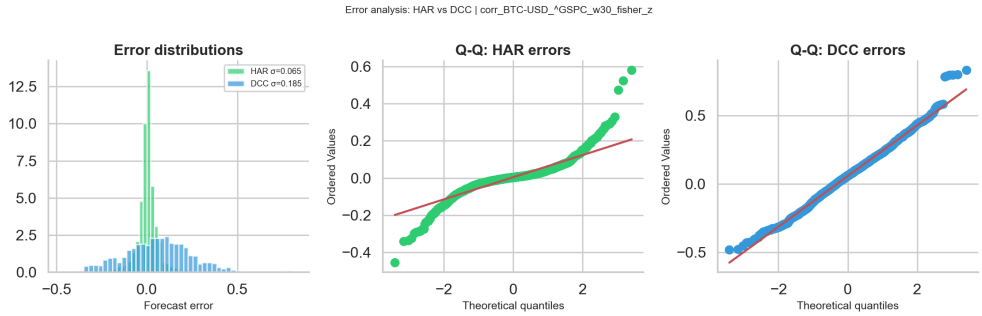

Figure 24: Forecast error distribution for the representative Bitcoin–S&P 500 dependency experiment.

B.4 Sensitivity by asset pair

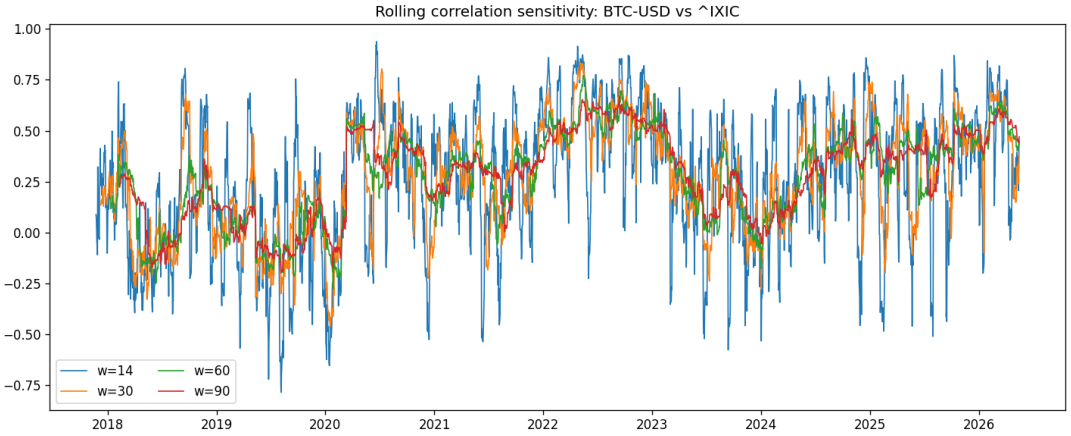

Figure 25: Sensitivity analysis for the Bitcoin–NASDAQ pair.

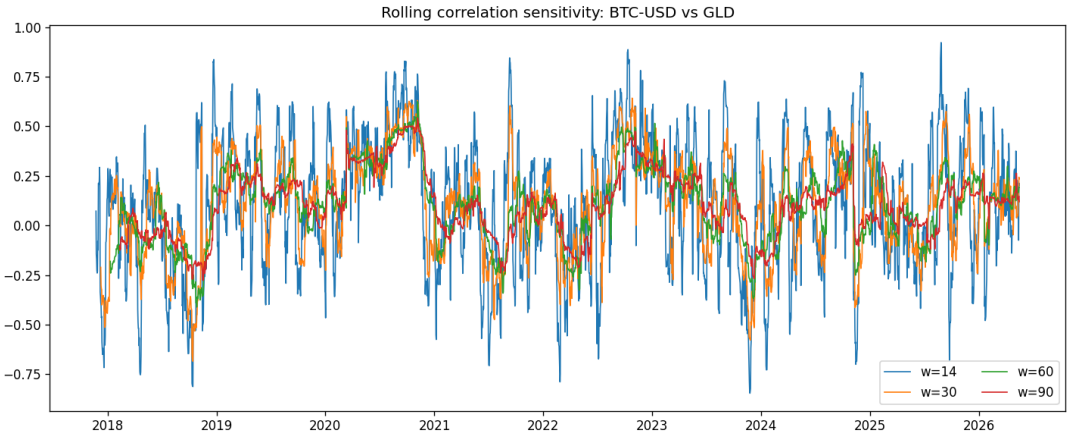

Figure 26: Sensitivity analysis for the Bitcoin–gold pair.

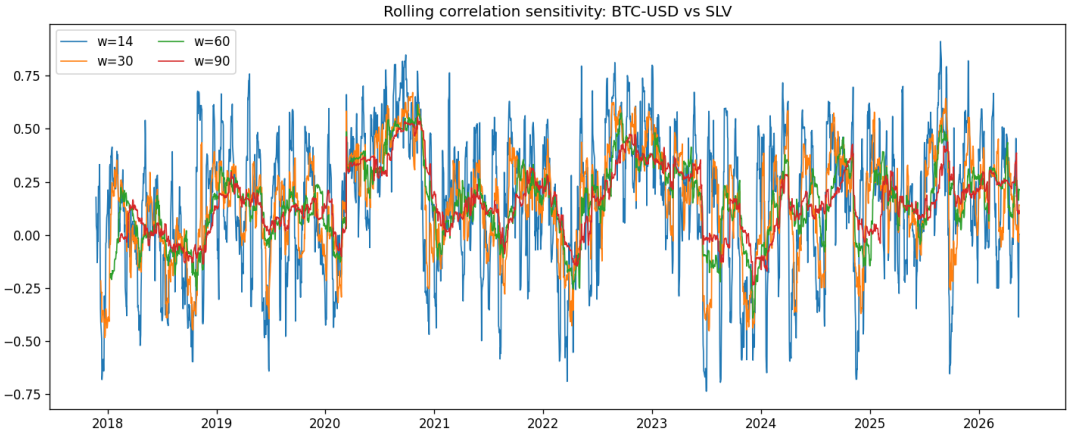

Figure 27: Sensitivity analysis for the Bitcoin–silver pair.

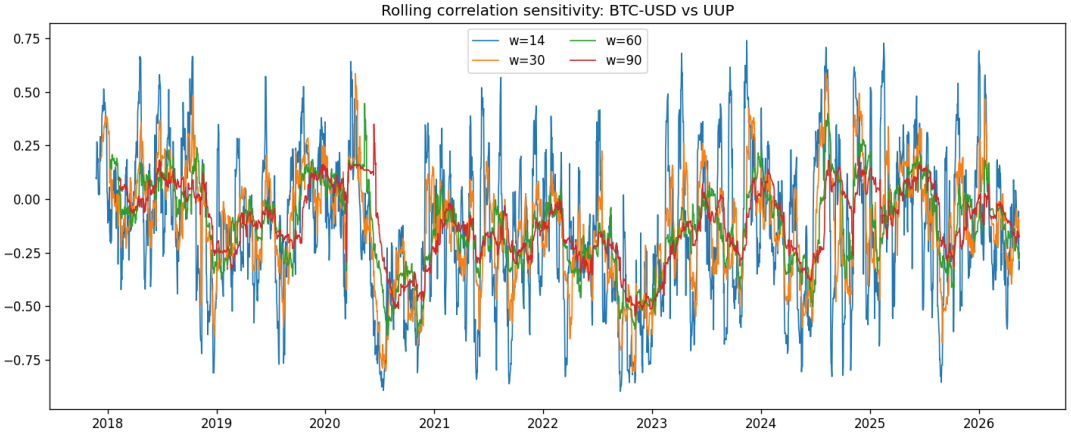

Figure 28: Sensitivity analysis for the Bitcoin–UUP pair.

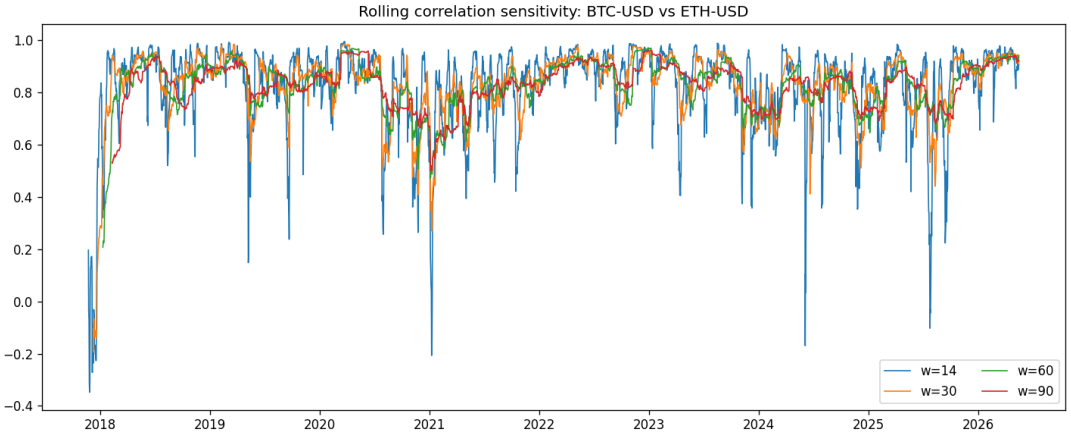

Figure 29: Sensitivity analysis for the Bitcoin–Ethereum pair.

B.5 Interpretation

The supplementary figures reinforce several points made in the main text. Forecast quality depends materially on the choice of rolling window, the target remains strongly persistence-driven, and performance differences across model families are easier to interpret through comparative visuals than through a single ranking statistic alone.

C Appendix C: Supplementary Signal-Layer Results

This appendix presents additional material for the investor-oriented warning layer.

C.1 Signal figures by asset

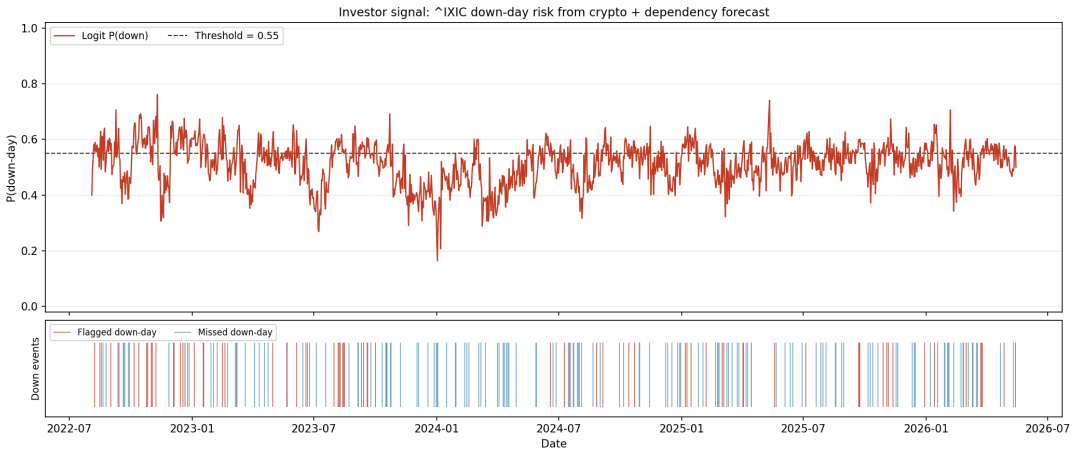

Figure 30: Signal-layer output for the Bitcoin–NASDAQ relation at the 30-day window.

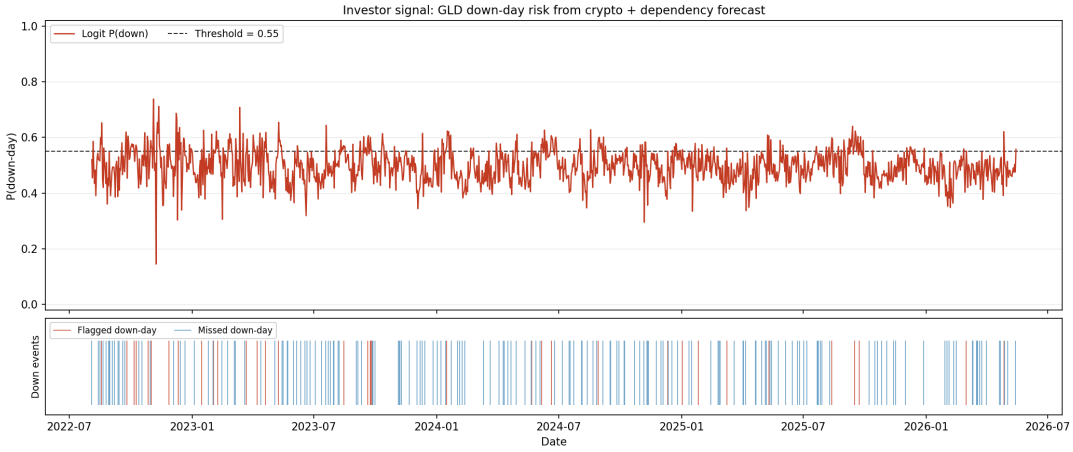

Figure 31: Signal-layer output for the Bitcoin–gold relation at the 30-day window.

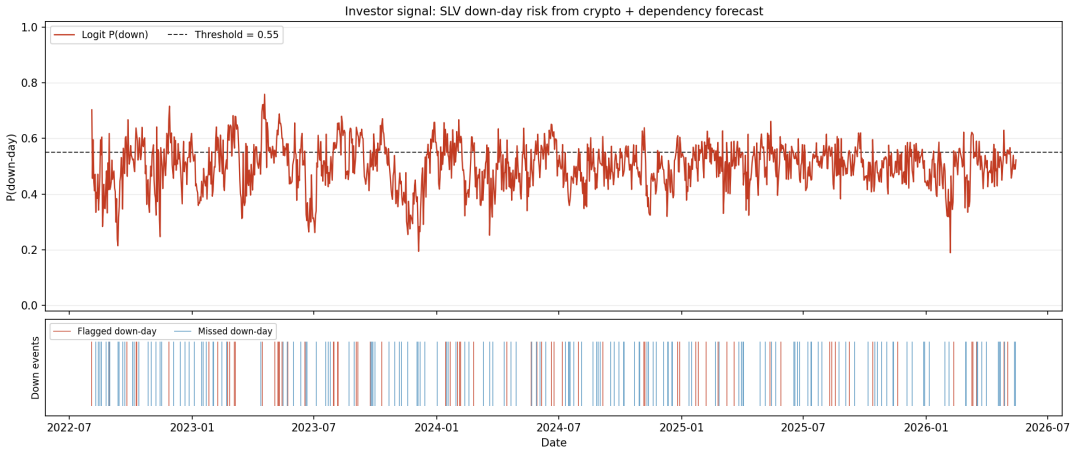

Figure 32: Signal-layer output for the Bitcoin–silver relation at the 30-day window.

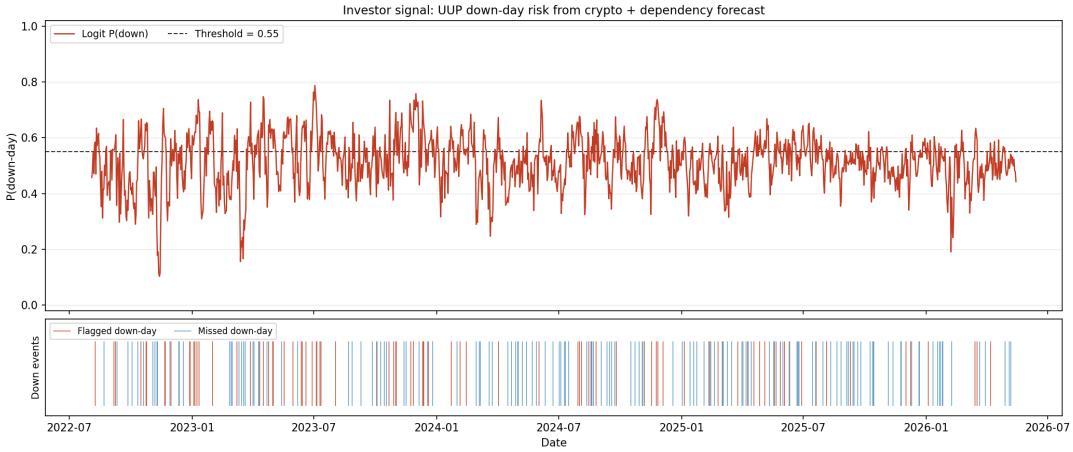

Figure 33: Signal-layer output for the Bitcoin–UUP relation at the 30-day window.

C.2 Cross-market warning interpretation

The supplementary signal figures show that the practical layer behaves differently across markets. The S&P 500 target displays the clearest warning episodes; the NASDAQ target is notably weaker, with several configurations falling below chance, likely because NASDAQ's sector concentration introduces idiosyncratic noise that Bitcoin's crypto-market information channel does not capture. Metal and dollar-linked targets are generally noisier but not uniformly uninformative. This heterogeneity supports the interpretation that the investor layer should be used selectively rather than uniformly.

C.3 Summary table

Table 6: Average signal-layer performance by classifier family.

Signal model	Balanced Accuracy	F1_down	AUC
Logit	0.506	0.207	0.510
GBM_Cls	0.513	0.178	0.517
RF_Cls	0.502	0.041	0.516

These summary values confirm that the signal layer is meaningful but modest. Its strongest role is as a supplementary warning mechanism rather than as a standalone allocator.

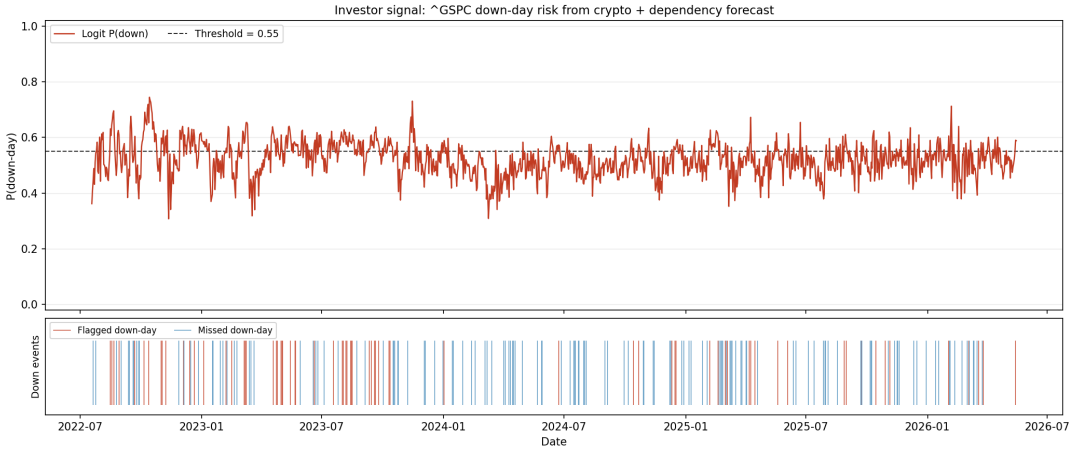

Figure 34: Supplementary S&P 500 warning signal at the 14-day window.

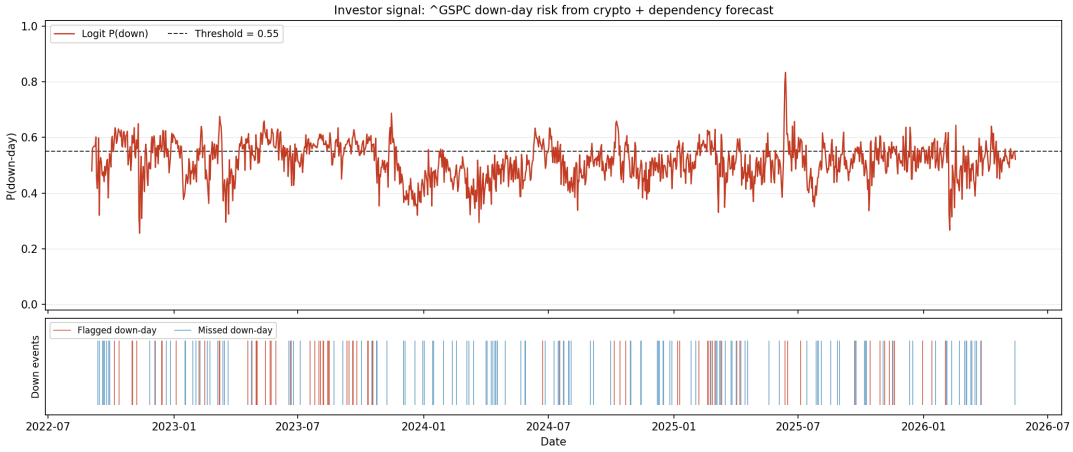

Figure 35: Supplementary S&P 500 warning signal at the 60-day window.

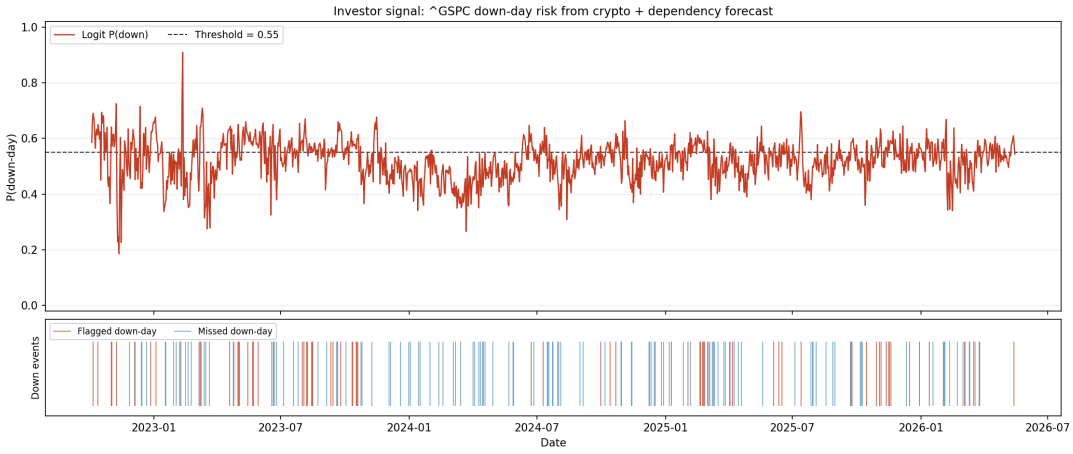

Figure 36: Supplementary S&P 500 warning signal at the 90-day window.

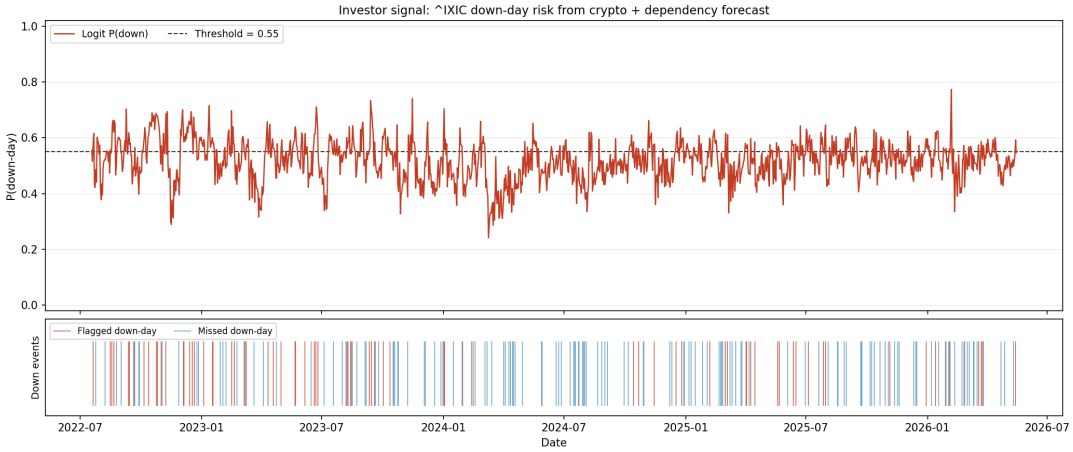

Figure 37: Supplementary NASDAQ warning signal at the 14-day window.

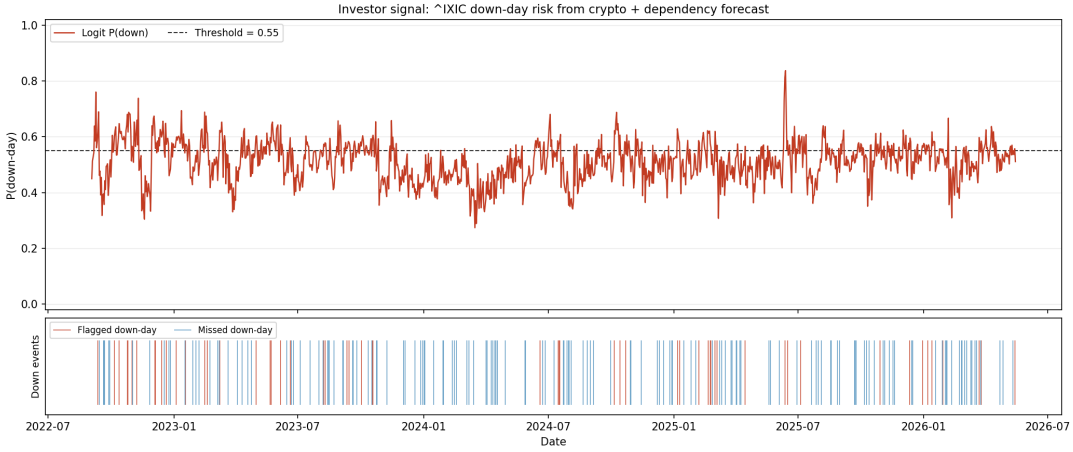

Figure 38: Supplementary NASDAQ warning signal at the 60-day window.

D Appendix D: Reproducibility and Code Structure

D.1 Project organization

The computational part of the thesis is structured as a reproducible research project rather than a collection of disconnected scripts. The main directories are organized as follows:

Listing 5: Top-level project directory structure.

```
master-thesis/
|-- main.py                 # main pipeline entry point
|-- run_all.py              # full automation: pipeline + notebooks + LaTeX
|-- config.yaml             # experiment configuration
|-- requirements.txt        # Python dependencies
|-- thesis_app/             # core Python modules
|   |-- pipeline.py         # data, features, walk-forward, metrics
|   |-- dcc_walk.py         # leakage-safe DCC-GARCH benchmark
|   |-- signal_layer.py     # investor stress-warning layer
|   |-- notebook_helpers.py # shared plotting and styling
|   |-- data_quality.py     # missing-value and outlier diagnostics
|   `-- regime_analysis.py  # historical regime catalogue
|-- notebooks/              # seven Jupyter notebooks (01-07)
|-- data/raw/               # cached downloaded price data
|-- data/processed/         # derived returns
|-- outputs/results/        # CSV summaries of experiments
|-- outputs/tables/         # LaTeX-exported tables
|-- outputs/figures/        # all generated figures
`-- thesis/                 # this document
```

D.2 Main implementation modules

The following modules constitute the core of the forecasting system:

- `pipeline.py`: orchestrates the full dependency-forecasting and signal-generation workflow.
- `dcc_walk.py`: contains the leakage-safe walk-forward DCC-GARCH implementation.
- `signal_layer.py`: defines the investor warning layer and associated classification metrics.
- `notebook_helpers.py`: shared plotting helpers, consistent styling, and analysis utilities used across all notebooks.
- `data_quality.py`: performs missing-value, outlier, and coverage diagnostics on the raw dataset.
- `regime_analysis.py`: implements the historical regime catalogue and regime-conditional statistics.

This separation of concerns is useful both scientifically and practically. Scientifically, it makes the logic of the workflow easier to audit. Practically, it allows individual modules

to be tested or improved without rewriting the entire system.

D.3 Analysis notebooks

Seven Jupyter notebooks in the `notebooks/` directory supplement the main pipeline with interactive diagnostics, publication-quality visualizations, and robustness analyses. Each notebook is self-contained and can be executed either from the command line or interactively inside Jupyter.

1. `01_EDA_Dataset.ipynb` — Exploratory data analysis. Produces normalized price trajectories, rolling volatility profiles, a pairwise correlation heatmap, rolling pairwise correlation time series, and the Fisher-transformation comparison plot. These figures appear in Appendix A.
2. `02_GridSearch.ipynb` — Hyperparameter search diagnostics. Reports leave-one-out cross-validation RMSE surfaces for the representative Bitcoin–S&P 500 experiment, regularization-path plots for Ridge and ElasticNet, and Random Forest feature-importance charts. Results support the model-selection discussion in Chapter 3.
3. `03_Model_Comparison.ipynb` — Model-ranking summary. Generates the average RMSE comparison bar chart, the R^2 comparison across window lengths, and the improvement-over-naive baseline figure across all asset pairs. These figures provide a visual complement to Table 3 in Chapter 4.
4. `04_DM_Tests_Visuals.ipynb` — Diebold–Mariano test visualization. Produces the DM heatmap comparing the best machine-learning model against DCC-GARCH (Figure 6) and the representative out-of-sample forecast panel for the Bitcoin–S&P 500 pair (Figure 5).
5. `05_XGB_vs_DCC.ipynb` — Error-profile diagnostics. Generates the rolling RMSE time series (Figure 7), forecast-error scatter plots, and error-distribution comparison histograms for the representative experiment.
6. `06_Regime_Analysis.ipynb` — Regime-conditional analysis. Produces the regime catalogue statistics table, regime-conditional RMSE bar charts, and data-quality summary figures. Provides the empirical basis for the regime-sensitivity discussion in Chapter 4.
7. `07_Robustness_Checks.ipynb` — Robustness and sensitivity analyses. Contains refit-frequency sensitivity sweeps, stress-threshold sensitivity curves, bootstrap confidence intervals for key metrics, signal-layer diagnostics, and the cross-asset Diebold–Mariano test comparing equity against non-equity pair performance.

All notebooks use a shared ROOT-detection routine that locates the project root by searching upward for `config.yaml`. This ensures correct path resolution whether a notebook is launched interactively from within the `notebooks/` directory or executed programmatically via `run_all.py` from the project root.

D.4 Execution flow

The full pipeline is launched through a single entry point that coordinates data acquisition, model training, notebook execution, and document compilation:

Listing 6: Automated full-pipeline runner (`run_all.py`), simplified for readability.

```
import argparse, subprocess, sys, os

BASE = os.path.dirname(os.path.abspath(__file__))
PYTHON = sys.executable

# Step 1: main pipeline (data, models, tables, figures)
subprocess.run([PYTHON, "main.py"], cwd=BASE, check=True)

# Step 2: execute each notebook with cwd set to project root
for nb in ["01_EDA_Dataset.ipynb", ..., "07_Robustness_Checks.ipynb"]:
    subprocess.run(
        [PYTHON, "-m", "jupyter", "nbconvert",
         "--to", "notebook", "--execute", "--inplace",
         f"--ExecutePreprocessor.cwd={BASE}",
         os.path.join(BASE, "notebooks", nb)],
        cwd=BASE
    )

# Step 3: run automated test suite (81 unit and integration tests)
subprocess.run(
    [PYTHON, "-m", "pytest", "tests/", "-v",
     "--junit-xml=outputs/results/test_results.xml"],
    cwd=BASE
)

# Step 4: compile the thesis PDF (test report table auto-included)
subprocess.run(
    ["latexmk", "-pdf", "-interaction=nonstopmode", "main.tex"],
    cwd=os.path.join(BASE, "thesis"), check=True
)
```

At a high level, the execution flow can be summarized as follows:

1. Load configuration and path settings.
2. Load or refresh raw price data from Yahoo Finance.
3. Compute synchronized log returns.
4. Generate rolling-correlation targets for each asset pair and window.
5. Build lagged and volatility-based feature matrices.
6. Fit all models in walk-forward mode and save predictions.
7. Compute metrics, DM tests, and export tables and figures.
8. Run the investor signal layer and export signal figures.

9. Execute seven Jupyter notebooks to produce supplementary diagnostics.
10. Run the automated `pytest` suite (81 unit and integration tests) and generate the test-report table for Appendix E.
11. Compile the LaTeX thesis document.

The main advantage of this design is that every figure and summary reported in the thesis can be traced back to a consistent computational procedure.

D.5 Configuration

Experiment parameters are stored in `config.yaml` and are read once at pipeline startup. This makes it straightforward to change the date range, asset universe, or window set without touching the source code:

Listing 7: Excerpt from `config.yaml` showing key experiment parameters.

```yaml
start_date: "2016-01-01"   # effective start: 2017-11-09 (ETH availability)
end_date:   null           # null = download up to today

assets:
  crypto:      ["BTC-USD", "ETH-USD"]
  traditional: ["^GSPC", "^IXIC", "GLD", "SLV", "UUP"]

base_asset:          "BTC-USD"
rolling_windows:     [14, 30, 60, 90]
forecast_horizon:    1
use_fisher_transform: true
min_train_size:      800
refit_every:         20
xgb_device:          "cuda"
random_state:        42
```

D.6 Verification performed during the project cleanup

During the refinement of the project, several implementation issues were corrected in order to align the code with thesis-level standards.

- A leakage-safe walk-forward DCC benchmark was introduced to avoid full-sample look-ahead contamination.
- The metrics pipeline was made consistent so that all relevant models, including DCC, are evaluated through the same summary path.
- Feature scaling was added for the regularized linear models.
- The XGBoost block was made robust to GPU failure through CPU fallback.
- The investor signal layer was integrated into the main pipeline and exported to dedicated tables and figures.
- Notebook imports and notebook-specific plotting logic were stabilized for presentation and interpretation.

These improvements matter because a thesis is evaluated not only by its conceptual idea but also by whether its empirical implementation is reliable, reproducible, and internally coherent.

D.7 How to rebuild the thesis artifacts

A practical benefit of the current project state is that the full workflow can be rerun from scratch with a single command from the project root:

Listing 8: Full rebuild from the project root directory.

```
python run_all.py
```

Alternatively, the three stages can be executed individually:

1. Run the main pipeline:
   `python main.py`
   This generates prediction CSVs, metrics, DM-test results, and the majority of figures.
2. Execute the notebooks in order:
   (a) `01_EDA_Dataset.ipynb` — dataset overview and EDA figures.
   (b) `02_GridSearch.ipynb` — cross-validated hyperparameter search.
   (c) `03_Model_Comparison.ipynb` — model-ranking table and heatmaps.
   (d) `04_DM_Tests_Visuals.ipynb` — DM-test heatmap and key forecast figure.
   (e) `05_XGB_vs_DCC.ipynb` — rolling-RMSE, scatter, and error-distribution figures.
   (f) `06_Regime_Analysis.ipynb` — regime catalogue and data-quality diagnostics.
   (g) `07_Robustness_Checks.ipynb` — refit sensitivity, stress threshold sensitivity, bootstrap confidence intervals, signal diagnostics, and cross-asset DM test.
3. Run the automated test suite and generate the appendix table:

   ```
   python -m pytest tests/ -v --junit-xml=outputs/results/test_results.xml
   ```

4. Rebuild the PDF document from the `thesis/` directory:

   ```
   pdflatex main.tex && biber main && pdflatex main.tex && pdflatex main.
       ↪ tex
   ```

Steps 1 and 2 must be completed before step 4 because several figures used in the thesis are produced by the notebooks. Step 3 must be completed before step 4 so that the test-report table (Appendix E) is up to date. This makes the written work maintainable even if the underlying market data are refreshed.

D.8 Final reproducibility note

The value of reproducibility in this thesis is not merely technical. Because the subject concerns financial forecasting, where accidental leakage and selective reporting are constant risks, a transparent and rerunnable codebase directly strengthens the credibility of the empirical conclusions.

E Appendix E: Additional Statistical Tests

This appendix presents three supplementary hypothesis tests that further validate the robustness of the empirical results: a Mincer-Zarnowitz forecast efficiency test, residual diagnostic tests (Ljung-Box and ARCH LM), and a Harvey-Leybourne-Newbold small-sample correction of the Diebold-Mariano statistic.

E.1 Mincer-Zarnowitz Forecast Efficiency Test

The Mincer-Zarnowitz (MZ) test [0] checks whether forecasts are unbiased and informationally efficient. For each model the auxiliary regression

$$y_t = \alpha + \beta\,\hat{y}_t + \varepsilon_t \tag{10}$$

is estimated by OLS and the joint null hypothesis H_0: $\alpha = 0,\ \beta = 1$ is tested with an F-statistic. Rejection ($p < 0.05$) indicates that the forecast is either biased ($\alpha \neq 0$) or fails to make full use of available information ($\beta \neq 1$).

Table 7 summarises the percentage of the 24 dependency–window experiments in which each model passes the MZ test, together with its average α and β coefficients.

Table 7: Mincer-Zarnowitz forecast efficiency: percentage of 24 experiments passing the joint F-test ($p \geq 0.05$) and average coefficient estimates.

Model	Efficient (%)	$\bar{\alpha}$	$\bar{\beta}$	$\bar{F}$
AR1	79.2	+0.0010	0.9976	2.1
Ridge	58.3	−0.0018	0.9970	6.6
RF	41.7	−0.0023	1.0029	8.3
GBM	16.7	−0.0060	1.0185	16.1
XGB_GPU	12.5	−0.0030	1.0173	17.0
ElasticNet	8.3	−0.0082	1.0223	19.8
Naive_Last	4.2	+0.0099	0.9702	17.1
DCC-GARCH	0.0	−0.0307	1.1780	291.0

AR1 passes the efficiency test in nearly four-fifths of experiments, confirming that it makes near-optimal use of the information set. Ridge is the most efficient among the ML models. DCC-GARCH never passes: its average $\bar{\beta} = 1.18$ reflects a systematic tendency to over-predict correlation magnitudes, consistent with the large positive RMSE penalty observed in the main results.

Table 8 provides the full detail for the representative pair BTC-USD vs S&P 500 at $w = 30$.

Table 8: Mincer-Zarnowitz test: BTC-USD vs S&P 500, $w = 30$. F-statistic tests H_0: $\alpha = 0, \beta = 1$; p_{MZ} is the associated p-value.

Model	$\hat{\alpha}$	$\hat{\beta}$	F	p_{MZ}	Efficient
AR1	+0.0071	0.9915	5.22	0.005	No
Ridge	+0.0140	0.9850	12.04	< 0.001	No
Naive_Last	+0.0115	0.9693	17.41	< 0.001	No
RF	+0.0184	0.9804	17.58	< 0.001	No
ElasticNet	+0.0091	1.0065	18.88	< 0.001	No
GBM	+0.0151	0.9998	26.32	< 0.001	No
XGB_GPU	+0.0181	1.0001	37.33	< 0.001	No
DCC-GARCH	−0.0814	1.4757	307.61	< 0.001	No

At this particular pair and window all models are technically rejected, yet the AR1 F-statistic (5.22) is an order of magnitude smaller than those for the ML models, and the DCC-GARCH statistic (307.6) stands apart as the most severely misspecified.

E.2 Residual Diagnostics: Ljung-Box and ARCH LM Tests

Two classical residual tests are applied to the OOS forecast errors $e_t = y_t - \hat{y}_t$ for every model × dependency × window combination (192 cases in total).

Ljung-Box Q-test (10 lags): tests H_0: no serial autocorrelation. Rejection indicates that the model has not fully captured the temporal structure of the target series.

ARCH LM test (5 lags): regresses e_t^2 on its own lags and tests H_0: no conditional heteroscedasticity. Rejection signals remaining volatility clustering in the residuals.

Table 9 reports, for each model, the percentage of experiments in which the residuals pass each test at the 5% level.

Table 9: Residual diagnostics: percentage of 24 experiments passing the Ljung-Box autocorrelation test and the ARCH LM heteroscedasticity test at $\alpha = 0.05$.

Model	No autocorr (%)	No ARCH (%)
Naive_Last	41.7	66.7
AR1	37.5	62.5
ElasticNet	0.0	0.0
Ridge	0.0	0.0
RF	0.0	0.0
GBM	0.0	0.0
XGB_GPU	0.0	0.0
DCC-GARCH	0.0	0.0

AR1 and Naive_Last partially capture the temporal structure of the series, passing the Ljung-Box test in roughly 37–42% of experiments. All ML models and DCC-GARCH leave strongly significant autocorrelation and volatility clustering in every experiment (0% pass rate). This confirms that persistence in the dependency series is the primary

driver of forecastability: any model that does not explicitly encode an autoregressive component fails to adequately model the temporal structure, and the large Q-statistics for tree-ensemble methods (e.g. Q = 1,225 for XGB_GPU at BTC-USD vs S&P 500, $w = 30$) underline that tree-based forecasters do not generalise the AR structure learned at training time.

E.3 Harvey-Leybourne-Newbold Corrected Diebold-Mariano Test

The standard Diebold-Mariano statistic uses the asymptotic standard-normal distribution, which tends to over-reject in finite samples. Harvey, Leybourne & Newbold [0] propose multiplying the DM statistic by the correction factor

$$k = \sqrt{\frac{n + 1 - 2h + h(h-1)/n}{n}}, \tag{11}$$

where n is the number of forecast observations and h the forecast horizon, and comparing the result to a $t(n-1)$ distribution. For $h = 1$ and $n >$ 1,000 the factor $k \approx 1.000$, so the practical effect on p-values is negligible.

Applied to all 240 DM comparisons (Table 10) the HLN correction produces zero sign changes in significance at the 5% level (average correction factor k = 1.000226). All conclusions regarding the relative ranking of models, and in particular the finding that every ML model significantly outperforms DCC-GARCH, are therefore robust to the small-sample correction.

Table 10: HLN-corrected DM test: ML models vs DCC-GARCH benchmark, BTC-USD vs S&P 500, $w = 30$. DM: original statistic (normal); DM_{HLN}: corrected statistic (t_{n-1}). All models beat DCC-GARCH at $p < 0.001$ under both tests.

Model	DM	p_{DM}	DM_{HLN}	p_{HLN}	Sig.
ElasticNet	25.80	< 0.001	25.79	< 0.001	***
Ridge	25.75	< 0.001	25.75	< 0.001	***
RF	26.67	< 0.001	26.66	< 0.001	***
GBM	27.38	< 0.001	27.37	< 0.001	***
XGB_GPU	27.39	< 0.001	27.38	< 0.001	***

E.4 Automated Validation Suite

In addition to the statistical tests above, the full project ships with an automated `pytest` suite of **81 unit and integration tests** organised into ten test classes. The suite is executed automatically as part of the reproducibility runner (`python run_all.py`) between the notebook step and the final LaTeX compilation, so every rebuilt PDF contains a fresh test report.

The test classes cover the following aspects of the pipeline:

- **Dataset Integrity** — raw price and return files exist, have correct tickers, positive prices, sorted and non-duplicate dates, and extend to at least 2026.

- **Data Quality** — missing-rate thresholds, zero-price detection, reasonable annualised volatility, pairwise coverage overlap.
- **Feature Engineering** — Fisher-z round-trip identity, rolling-correlation range, no look-ahead in prediction CSVs.
- **Model Metrics** — RMSE > 0, $R^2 \leq 1$, MAE $\leq$ RMSE, all windows and expected models present, AR1 beats DCC-GARCH on average.
- **DM Tests** — finite DM statistics, p-values in $[0, 1]$, correct sign convention for Ridge vs DCC-GARCH.
- **Signal Metrics** — balanced accuracy and F1 in $[0, 1]$, AUC above random, exit rate within a sensible range.
- **Walk-Forward** — correct number of prediction CSVs, non-trivial predictions, index alignment with returns.
- **Output Files** — all expected CSV, JSON, PNG, and `.tex` artefacts are present after a full pipeline run.
- **Bootstrap CI** — lower $\leq$ mean $\leq$ upper for RMSE and R^2; CI width below 0.10.
- **Sensitivity** — all five `refit_every` values present; RMSE in a physically plausible range.

Table 11 lists each individual test together with its result and wall-clock execution time. All 81 tests pass on the present outputs; skipped tests indicate that the optional `arch` package (DCC-GARCH) is not installed in the current environment.

Table 11: Automated validation suite results

Run `python run_all.py` to regenerate this table.

References

[0] François Longin and Bruno Solnik, „Extreme Correlation of International Equity Markets", in: *The Journal of Finance* 56.2 (2001), pp. 649–676.

[0] Elie Bouri et al., „On the hedge and safe haven properties of Bitcoin: Is it really more than a diversifier?", in: *Finance Research Letters* 20 (2017), pp. 192–198.

[0] Shaen Corbet et al., „Exploring the dynamic relationships between cryptocurrencies and other financial assets", in: *Economics Letters* 165 (2018), pp. 28–34.

[0] Qiang Ji et al., „Network causality structures among Bitcoin and other financial assets", in: *The Quarterly Review of Economics and Finance* 70 (2018), pp. 203–213.

[0] Ladislav Kristoufek, „What are Bitcoin's main drivers?", in: *PLOS ONE* 10.4 (2015), e0123923.

[0] Anne Haubo Dyhrberg, „Bitcoin, gold and the dollar – A GARCH volatility analysis", in: *Finance Research Letters* 16 (2016), pp. 85–92.

[0] Tony Klein, Hien Pham Thu, and Thomas Walther, „Bitcoin is not the new gold – A comparison of volatility, correlation, and portfolio performance", in: *International Review of Financial Analysis* 59 (2018), pp. 105–116.

[0] Andrew J. Patton, „Modelling asymmetric exchange rate dependence", in: *International Economic Review* 47.2 (2006), pp. 527–556.

[0] Robert F. Engle and Kevin Sheppard, „Theoretical and empirical properties of dynamic conditional correlation multivariate GARCH", in: *NBER Working Paper* 8554 (2001).

[0] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models", in: *Journal of Business & Economic Statistics* 20.3 (2002), pp. 339–350.

[0] Fulvio Corsi, „A simple approximate long-memory model of realized volatility", in: *Journal of Financial Econometrics* 7.2 (2009), pp. 174–196.

[0] Torben G. Andersen et al., „Modeling and forecasting realized volatility", in: *Econometrica* 71.2 (2003), pp. 579–625.

[0] Hui Zou and Trevor Hastie, „Regularization and Variable Selection via the Elastic Net", in: *Journal of the Royal Statistical Society: Series B* 67.2 (2005), pp. 301–320.

[0] Leo Breiman, „Random Forests", in: *Machine Learning* 45.1 (2001), pp. 5–32.

[0] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine", in: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232.

[0] Tianqi Chen and Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System", in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[0] Francis X. Diebold and Roberto S. Mariano, „Comparing Predictive Accuracy", in: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.

[0] yfinance developers, *yfinance Documentation*, 2026, `https://ranaroussi.github.io/yfinance/` (visited on 03/10/2026).

[0] Marcos Lopez de Prado, *Advances in Financial Machine Learning*, Wiley, 2018.

[0] Ivo Welch and Amit Goyal, „A Comprehensive Look at the Empirical Performance of Equity Premium Prediction", in: *The Review of Financial Studies* 21.4 (2008), pp. 1455–1508.

[0] Jacob Mincer and Victor Zarnowitz, „The Evaluation of Economic Forecasts", in: *Economic Forecasts and Expectations*, ed. by Jacob Mincer, National Bureau of Economic Research, 1969, pp. 1–46.

[0] David Harvey, Stephen Leybourne, and Paul Newbold, „Testing the equality of prediction mean squared errors", in: *International Journal of Forecasting* 13.2 (1997), pp. 281–291.

[0] Fakultet inženjerskih nauka Univerziteta u Kragujevcu, *Master radovi*, 2026, `https://www.cis.kg.ac.rs/master/` (visited on 03/10/2026).

[0] elektrotehnika, *finlatex: finthesis dokument klasa*, 2026, `https://github.com/elektrotehnika/finlatex` (visited on 03/10/2026).

[0] scikit-learn developers, *scikit-learn User Guide*, 2026, `https://scikit-learn.org/stable/user_guide.html` (visited on 03/10/2026).

[0] XGBoost developers, *XGBoost Documentation*, 2026, `https://xgboost.readthedocs.io/en/stable/` (visited on 03/10/2026).

[0] arch developers, *arch Documentation*, 2026, `https://bashtage.github.io/arch/` (visited on 03/10/2026).

[0] Shihao Gu, Bryan Kelly, and Dacheng Xiu, „Empirical asset pricing via machine learning", in: *The Review of Financial Studies* 33.5 (2020), pp. 2223–2273.

[0] Satoshi Nakamoto, „Bitcoin: A peer-to-peer electronic cash system", in: *Decentralized Business Review* (2008).

[0] Greta M. Ljung and George E. P. Box, „On a measure of lack of fit in time series models", in: *Biometrika* 65.2 (1978), pp. 297–303.

[0] Robert F. Engle, „Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation", in: *Econometrica* 50.4 (1982), pp. 987–1007.

Објављени радови

Кандидат није објавио радове током израде мастер рада.